

QUADRASHAP: EXACT SHAPLEY VALUES FOR PRODUCT GAMES IN $O(\log d)$ PARALLEL TIME

Anonymous authors

Paper under double-blind review

ABSTRACT

We introduce QUADRASHAP, a numerically stable, parallelisable method for exact Shapley computation in *product games*, cooperative games whose coalition values factorise over players. We show that baseline and empirical interventional value functions are product games or weighted sums thereof across a broad model class: generalised linear and additive models with a log link, Cox proportional-hazards models, logistic regression and naive Bayes on the odds scale, product-kernel machines, and rule ensembles, including decision trees. For a d -player product game, we express each Shapley value as an integral over $[0, 1]$ of a polynomial of degree at most $d - 1$, replacing the exponentially large coalition sum. Gauss–Legendre quadrature is therefore exact with $m_q \geq \lceil d/2 \rceil$ nodes; below this threshold, we prove geometric error decay in m_q . Our implementation combines log-space product evaluation with sign tracking and associative reductions, computing all d Shapley values in $O(dm_q)$ work and $O(\log d)$ parallel time per product game. The cost scales linearly with the number of terms in a weighted sum. Experiments on tree ensembles and product-kernel models, two of the most popular models in practice, show that errors fall by several orders of magnitude within tens of nodes, with a few hundred nodes yielding accurate approximations for models with thousands of features. QUADRASHAP remains numerically stable where existing methods break down and is faster than all stable baselines at large problem sizes. Code is available at <https://anonymous.4open.science/r/quadrashap-28D7>.

1 INTRODUCTION

Shapley values (Shapley, 1953) are now foundational for explaining predictions of machine learning (ML) models, yielding feature attributions with strong axiomatic guarantees rooted in cooperative game theory (Lundberg & Lee, 2017). Their principled nature has driven widespread adoption in explainable AI, spanning both model-agnostic methods (Lundberg & Lee, 2017; Muschalik et al., 2024; Fumagalli et al., 2026) and model-specific methods that exploit the structure of particular model classes, e.g., TreeSHAP for tree ensembles (Lundberg et al., 2020; Yu et al., 2022a), GPSHAP for Gaussian processes (Chau et al., 2023; Mohammadi et al., 2026), and RKHS-SHAP for kernel methods (Chau et al., 2022). Shapley-based attribution has thus become a central paradigm for interpreting modern ML systems.

While popular, Shapley values are generally expensive to compute exactly, as their definition involves a sum over exponentially many feature coalitions. Model-specific methods can circumvent this combinatorial cost by exploiting the structure of the model being explained, yielding exact or near-exact values in polynomial time. We build on this approach by focusing on a unifying algebraic structure that cuts across model classes: *multiplicative factorisation* of the coalition value function. This structure is captured by *product games* (Dehez, 2024), whose coalition values take the form $v(S) = \prod_{j \in S} u_j$ for player-specific factors u_j (Definition 1). By linearity of the Shapley value, this formulation extends directly to weighted sums of product games. As we show in Table 1, this representation encompasses a broad range of widely used models and attribution settings, including generalised linear and additive models with a log link, Cox proportional-hazards models, logistic regression and naive Bayes on the odds scale, product-kernel machines, and rule ensembles, including decision trees.

Despite arising from seemingly different models, these coalition value functions share the same algebraic form: a weighted sum of products of feature-wise factors. Existing methods have largely exploited this structure separately in different ways within individual model families, leading to distinct notation, derivations, and algorithms for each setting (Lundberg et al., 2020; Yu et al., 2022a; Chau et al., 2022; 2023; Mohammadi et al., 2025; Li et al., 2026). We instead ask whether the Shapley computation itself can be abstracted into a single reusable primitive, independently of how the underlying coalition game is constructed. Such a separation allows the model-specific derivation to terminate once the value function has been expressed as a sum of products, after which a common algorithm and numerical analysis apply. We show that product games provide precisely this abstraction: whenever a predictive model admits a tractable sum-of-products representation, its empirical interventional value function can be expressed exactly as a weighted sum of product games, with analogous representations for the baseline and neutral-factor value functions (Section 4).

It remains to compute the Shapley values of a product game both efficiently and numerically reliably. Two distinct challenges arise. First, the defining coalition sum contains exponentially many terms. Existing approaches can avoid this enumeration by recovering the coefficients of the associated polynomial through interpolation (Yu et al., 2022a), but this becomes increasingly ill-conditioned as the polynomial degree grows (Appendix E.3). Second, irrespective of how the coalition sum is evaluated, direct multiplication of thousands of feature-wise factors quickly exceeds the dynamic range of floating-point arithmetic. Thus, scaling product-game attribution to high dimensions requires addressing not only computational complexity, but also numerical stability.

While this work was under development, TreeGrad-Shap (Li et al., 2026) introduced Gauss–Legendre quadrature for computing Shapley values in path-dependent decision trees, with Quadrature-TreeSHAP (Wettenstein et al., 2026) appearing concurrently. When specialised to path-dependent trees, our formulation recovers the same leaf-wise quadrature integrand and evaluates it at the same Gauss–Legendre nodes. Our contribution lies instead in elevating this observation from a tree-specific construction to a general computational primitive for product games. This abstraction immediately extends the same quadrature machinery across the model classes described in Section 4, while also exposing challenges that are less pronounced for shallow tree paths: approximation at degrees determined by the number of features, numerical stability of long products, and efficient parallel evaluation. We address these through a geometric approximation guarantee below the exactness threshold, log-space evaluation with sign tracking, and an associative-reduction formulation for parallel computation. A detailed comparison with the tree-specific quadrature methods is given in Section A.

Contributions. Our contributions are threefold.

First, we reduce Shapley computation for product games to one-dimensional Gaussian quadrature, with exactness and controlled approximation guarantees. We introduce QUADRASHAP—for *quadrature-based Shapley value computation*—based on an integral representation obtained from the Beta-function form of the Shapley weights (Section 3). This representation collapses the exponentially large coalition sum to the integral over $[0, 1]$ of a polynomial of degree at most $d - 1$, avoiding both subset enumeration and coefficient recovery. Gauss–Legendre quadrature is exact for $m_q \geq \lceil d/2 \rceil$ nodes; below this threshold, we prove geometric decay of the approximation error in m_q , providing a principled accuracy–computation trade-off (Section 5.1).

Second, we compute all d Shapley values exactly in $O(\log d)$ parallel time. At each quadrature node, all d leave-one-out products are obtained from shared log-space reductions with sign tracking, avoiding the underflow and overflow that arise from direct multiplication of many feature-wise factors (Section 3.2). The resulting algorithm computes all Shapley values in $O(d m_q)$ work and, through associative reductions, $O(\log d)$ parallel time (Proposition 3).

Third, we establish product games as a reusable computational primitive across a broad class of predictive models. Any model admitting a tractable sum-of-products representation induces an exact weighted sum of product games under the empirical interventional value function, with analogous reductions for the baseline and neutral-factor value functions (Section 4). Consequently, the same QUADRASHAP algorithm and numerical analysis apply across otherwise distinct attribution problems, including log-link regression models, Cox proportional-hazards models, odds-scale classifiers, product-kernel machines, and rule ensembles such as decision trees.

2 PRELIMINARIES

Notation. Let $\mathcal{D}$ denote the set of d input features and $2^{\mathcal{D}}$ its power set. The training set consists of n samples $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \mathbb{R}$ (or a discrete label set in classification). We write $\mathbf{X} \in \mathbb{R}^{n \times d}$ for the feature matrix. For a subset $\mathcal{S} \subseteq \mathcal{D}$, $\mathbf{X}_{\mathcal{S}}$ denotes the restriction of $\mathbf{X}$ to the columns indexed by $\mathcal{S}$, and $\mathbf{x}_{\mathcal{S}}$ the restriction of a vector $\mathbf{x}$ to the same subset. We use bold lowercase letters for vectors, bold uppercase letters for matrices, calligraphic letters for sets, and capital letters for random variables. Element-wise multiplication and division are denoted by $\odot$ and $\oslash$, respectively, and expectation by $\mathbb{E}$. All proofs are deferred to Appendix D.

Shapley values. The Shapley value (Shapley, 1953) is a classical solution concept from cooperative game theory and now popular for feature attribution in explainable ML. It assigns an importance score to each feature by averaging its marginal contribution over all coalitions, with weights determined by coalition size. Among allocation rules, it is uniquely characterized by the axioms of *efficiency*, *symmetry*, the *null-player* property, and *linearity* (Shapley, 1953), and these axioms have been repeatedly invoked as motivation (Merrick & Taly, 2020) for using Shapley values in explainable ML.

Formally, given a value function $v : 2^{\mathcal{D}} \rightarrow \mathbb{R}$ that assigns a scalar as a contribution to each feature subset, the Shapley value of feature $j \in \mathcal{D}$ is defined as

$$\phi_j(v) = \sum_{\mathcal{S} \subseteq \mathcal{D} \setminus \{j\}} \mu(|\mathcal{S}|)(v(\mathcal{S} \cup \{j\}) - v(\mathcal{S})), \quad \mu(s) = \frac{s!(d-s-1)!}{d!}. \quad (1)$$

The term $v(\mathcal{S} \cup \{j\}) - v(\mathcal{S})$ is the marginal contribution of feature j to coalition $\mathcal{S}$, and we denote μ as the *Shapley weights*. Equation (1) sums over 2^{d-1} coalitions, so exact computation is exponential in d for generic v . Scalable algorithms have therefore been developed along two complementary directions: *model-agnostic* approximations such as KernelSHAP and its refinements (Lundberg & Lee, 2017; Muschalik et al., 2024; Fumagalli et al., 2026), and *model-specific* algorithms that exploit algebraic or combinatorial structure in v (Lundberg et al., 2020; Yu et al., 2022a; Merrick & Taly, 2020; Yang, 2021; Jiang et al., 2025; Mohammadi et al., 2025; 2026; Chau et al., 2022; 2023). The methods developed in this paper belong to the model-specific line and target the broad class of value functions with *multiplicative* structure.

Value functions for explainability. In explainable ML, the value function v specifies how a predictive model f is evaluated when only a subset of features is treated as “present,” and different choices correspond to different semantics for missing features (Sundararajan & Najmi, 2020; Molnar, 2022). The two standard choices are the *observational/conditional* value $v_{\mathbf{x}}^{\text{obs}}(\mathcal{S}) = \mathbb{E}[f(X) \mid X_{\mathcal{S}} = \mathbf{x}_{\mathcal{S}}]$, which imputes missing features from the conditional distribution, and the *interventional/marginal* value $v_{\mathbf{x}}^{\text{int}}(\mathcal{S}) = \mathbb{E}_{X_{\mathcal{D} \setminus \mathcal{S}}}[f(\mathbf{x}_{\mathcal{S}}, X_{\mathcal{D} \setminus \mathcal{S}})]$, which treats missing features as drawn from their marginal distribution independently of $\mathbf{x}_{\mathcal{S}}$ (Janzing et al., 2020; Chen et al., 2020; Heskes et al., 2020). Our interest is in value functions that factorize across features, namely product games (Definition 1) and their weighted sums. This structure is not an assumption we place on v : we show in Section 4 that, for every model with a sum-of-products representation (Definition 2), the interventional value function evaluated over an empirical background sample, and the baseline and neutral-factor value functions as special cases, are exactly weighted sums of product games.

Product games. A *product game* is a cooperative game whose coalition value factorizes across players (Dehez, 2024).

Definition 1 (Product game). *With the convention $v(\emptyset) = 1$, $v : 2^{\mathcal{D}} \rightarrow \mathbb{R}$ is a product game if $v(\mathcal{S}) = \prod_{j \in \mathcal{S}} u_j(x_j)$ for $\mathcal{S} \subseteq \mathcal{D}$, where $u_j(x_j)$ denotes the multiplicative contribution associated with feature j for the instance $\mathbf{x}$.*

We write $u_j := u_j(x_j)$ throughout for simplicity. Substituting this form into Equation (1), the marginal contribution of feature i to a coalition $\mathcal{S} \subseteq \mathcal{D} \setminus \{i\}$ yields $v(\mathcal{S} \cup \{i\}) - v(\mathcal{S}) = (\prod_{j \in \mathcal{S}} u_j)(u_i - 1)$, so the Shapley value reduces to

$$\phi_i(v) = (u_i - 1) \sum_{\mathcal{S} \subseteq \mathcal{D} \setminus \{i\}} \mu(|\mathcal{S}|) \prod_{j \in \mathcal{S}} u_j, \quad \mu(s) = \frac{s!(d-s-1)!}{d!}. \quad (2)$$

The remainder of the paper develops an efficient algorithm for computing Shapley values of product games and shows that this multiplicative structure arises naturally in several ML models.

Gauss–Legendre quadrature. Gauss–Legendre quadrature (Davis & Rabinowitz, 1984) approximates integrals over a bounded interval via a weighted sum of function evaluations at carefully chosen nodes. On $[0, 1]$, it gives

$$\int_0^1 g(t) dt \approx \sum_{q=1}^{m_q} \omega_q g(\tau_q),$$

where the nodes $\{\tau_q\}_{q=1}^{m_q} \subset (0, 1)$ are the roots of the shifted Legendre polynomial of degree m_q , and the weights $\{\omega_q\}$ are computed once via the Golub–Welsch algorithm (Golub & Welsch, 1969). The rule has two key properties that drive our approach: (i) with m_q nodes it is *exact* for every polynomial of degree at most $2m_q - 1$, and (ii) when exactness is not required, smaller values of m_q provide a controlled approximation to the integral (Trefethen, 2008).

3 SHAPLEY VALUES FOR PRODUCT GAMES VIA GAUSS–LEGENDRE QUADRATURE

The core challenge in evaluating equation (2) is the weighted sum over all 2^{d-1} subsets of $\mathcal{D} \setminus \{i\}$. Rather than recovering this sum via interpolation (Yu et al., 2022a; Merrick & Taly, 2020) or recursive coefficient computation (Mohammadi et al., 2025), we show that the Shapley weights can be absorbed directly into a one-dimensional integral.

3.1 A QUADRATURE-READY SHAPLEY FORMULA FOR PRODUCT GAMES

We now present a one-dimensional integral representation that can yield an analytic formula for $\phi_i(v)$ and enables stable and efficient computation via Gauss–Legendre quadrature.

Proposition 1. *Let $v_{\mathbf{x}}(S) = \prod_{j \in S} u_j$ be a product game. Then, for each $i \in \mathcal{D}$,*

$$\phi_i(v) = (u_i - 1) \int_0^1 \prod_{j \neq i} ((1 - t) + t u_j) dt. \quad (3)$$

Equation (3) absorbs the Shapley weights into a one-dimensional integral, with the reduction relying on the Beta-function representation $\mu(s) = \int_0^1 t^s (1 - t)^{d-s-1} dt$ of the Shapley weights. The same identity follows from Owen’s multilinear extension of the game (Owen, 1972), which is the route taken for decision trees by TreeGrad-Shap (Li et al., 2026); Section A compares the approaches. The remaining question is how to evaluate this integral efficiently. We next show that the polynomial structure of the integrand makes Gauss–Legendre quadrature an exact match: with sufficiently many nodes, the quadrature sum recovers the integral exactly, yielding an efficient and exact algorithm.

Proposition 2. *Let $v(S) = \prod_{j \in S} u_j$ be a product game and let $\{(\tau_q, \omega_q)\}_{q=1}^{m_q}$ be Gauss–Legendre nodes and weights on $[0, 1]$. Define, for each $i \in \mathcal{D}$,*

$$\hat{\phi}_i(v) := (u_i - 1) \sum_{q=1}^{m_q} \omega_q \prod_{j \neq i} ((1 - \tau_q) + \tau_q u_j). \quad (4)$$

1. (Exactness.) *If $m_q \geq \lceil d/2 \rceil$, then $\hat{\phi}_i(v) = \phi_i(v)$ for all $i \in \mathcal{D}$.*
2. (Approximation.) *For $m_q < \lceil d/2 \rceil$, let E_ρ be a Bernstein ellipse (Trefethen, 2008) for $[0, 1]$ with parameter $\rho > 1$, and set $M_\rho := \max_{z \in E_\rho} |g_i(z)|$ where $g_i(z) := \prod_{j \neq i} ((1 - z) + z u_j)$. Then, for C_ρ dependent only on ρ ,*

$$|\hat{\phi}_i(v) - \phi_i(v)| \leq |u_i - 1| \cdot C_\rho M_\rho \rho^{-2m_q}.$$

Beyond exactness, Proposition 2 changes the nature of the computation: the sum over 2^{d-1} coalitions becomes m_q evaluations of a product of $d-1$ factors, with no coalition enumerated and no polynomial coefficient recovered. The latter is what interpolation-based methods (Yu et al., 2022a; Heidari et al., 2026) do by inverting a Vandermonde system, whose condition number grows exponentially in the degree (Gautschi, 1975), so the recovered coefficients lose accuracy as d grows; equation (4) is a forward sum with positive weights and inverts nothing, so this source of instability is absent. Below the exactness threshold the error decays geometrically in m_q (Section 5.1). Evaluating the $d m_q$ leave-one-out products efficiently and within floating-point range is the subject of the next subsection.

3.2 STABLE AND EFFICIENT SHAPLEY ATTRIBUTION

Stable evaluation via shared log-space products. A direct implementation of equation (4) forms a leave-one-out product of $d - 1$ factors for every pair (i, q) , at $O(d^2 m_q)$ operations, and when d is large or some u_j are far from one these products overflow or underflow in floating-point arithmetic. Both problems have one remedy, because the d leave-one-out products at a node are a single full product with one factor removed: we form that product once per node, in log-space with sign tracking, and recover every leave-one-out product by a division. For each node τ_q , define the per-feature factors and their full product

$$T_{q,j} := (1 - \tau_q) + \tau_q u_j, \quad P_q := \prod_{j=1}^d T_{q,j},$$

so that every leave-one-out product reduces to a single division, $\prod_{j \neq i} T_{q,j} = P_q / T_{q,i}$. This collapses the (i, q) -loop from $O(d^2 m_q)$ to $O(d m_q)$. To execute the division stably, we accumulate the log-magnitude of P_q and track its sign separately,

$$\log |P_q| = \sum_{j=1}^d \log |T_{q,j}|, \quad \text{sgn}(P_q) = \prod_{j=1}^d \text{sgn}(T_{q,j}), \quad (5)$$

so that the Shapley estimator takes the numerically stable form

$$\hat{\phi}_i(v) = (u_i - 1) \sum_{q=1}^{m_q} \omega_q \text{sgn}(P_q) \text{sgn}(T_{q,i}) \exp(\log |P_q| - \log |T_{q,i}|). \quad (6)$$

When all u_j are nonnegative, as for distance-based kernels and likelihood factors, the sign book-keeping is vacuous; it is retained so that equation (6) remains correct when some u_j are negative. Zero factors $T_{q,j}$ must be handled separately, since $\log |T_{q,j}|$ is undefined at zero; Section B gives a division-free variant of the evaluator that accommodates them.

Parallel implementation and complexity. An *associative scan* is a fundamental parallel building block: given a length- d sequence and an associative binary operator $\oplus$, it computes all prefix reductions $a_1, a_1 \oplus a_2, \dots, a_1 \oplus \dots \oplus a_d$ simultaneously (Blelloch, 1990). By pairing elements in a binary tree of depth $\log d$, this can be done in $O(d)$ total work and $O(\log d)$ parallel time, and the same tree structure trivially yields the full *associative reduction* $a_1 \oplus \dots \oplus a_d$ as a special case.

The log-space quantities $\log |P_q|$ and $\text{sgn}(P_q)$ are reductions of associative operators, i.e., addition and multiplication, over the feature index j , and are therefore direct applications of this primitive. The same pattern reappears in the final weighted sum over quadrature nodes in equation (6), which is itself an associative reduction over m_q terms. Since the d Shapley values are independent given $\{(\log |P_q|, \text{sgn}(P_q))\}_{q=1}^{m_q}$, they are computed in parallel across i at no additional asymptotic cost. This analysis establishes the following result.

Proposition 3. *Let d be the number of features and m_q the number of Gauss–Legendre nodes.*

1. (Sequential.) *The estimator $\{\hat{\phi}_i\}_{i=1}^d$ in equation (6) requires $O(d m_q)$ total work.*
2. (Parallel.) *The parallel time complexity is $O(\log d)$ with $O(d m_q / \log d m_q)$ processors.*

In addition to the time complexity, memory requirements depend on how the product-game components are constructed and accumulated. Section 4 examines this issue for empirical interventional value functions and describes a blockwise implementation that avoids materializing all component-background factor tables simultaneously.

4 MULTIPLICATIVE MODELS AND THEIR INDUCED PRODUCT GAMES

The previous section developed an exact, numerically stable algorithm for a single product game. We now show that models with a multiplicative representation across features induce value functions, interventional as well as baseline, that are weighted sums of product games.

4.1 THE REDUCTION TO PRODUCT GAMES

Definition 2. A predictive model $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is multiplicative with p components if

$$f(\mathbf{x}) = \sum_{r=1}^p c_r \prod_{j \in \mathcal{D}} u_j^{(r)}(x_j), \quad (7)$$

for coefficients $c_r \in \mathbb{R}$ and univariate factor functions $u_j^{(r)} : \mathbb{R} \rightarrow \mathbb{R}$.

Each index r identifies a multiplicative component, formed as a product of per-feature terms. When $p = 1$, the model is a single product, while $p > 1$ gives a finite sum of such products. The next subsection applies this representation to several model classes, where p is one, the number of training points, or the number of rules.

The Shapley values of f depend on how absent features are treated. For a multiplicative model the natural choices all act factor by factor: the factor $u_j^{(r)}(x_j)$ of an absent feature is replaced by an *absent factor* $\tilde{u}_j^{(r)}$ that does not depend on $\mathbf{x}_S$, so the coalition value remains a weighted sum of products.

The simplest choice is the *neutral factor* $\tilde{u}_j^{(r)} \equiv 1$, which removes the absent feature from every product and has been used to define Shapley values for product-kernel machines (Mohammadi et al., 2025); the resulting coalition value is a weighted sum of product games in the sense of Definition 1. A second choice fixes a reference point $\mathbf{x}^b$ and takes $\tilde{u}_j^{(r)} = u_j^{(r)}(x_j^b)$, which gives the *baseline* value function $v_{\mathbf{x}}(\mathcal{S}) = f(\mathbf{x}_S, \mathbf{x}_{\mathcal{D} \setminus S}^b)$. Our focus is the interventional value function of Section 2, which averages the baseline value over the distribution of the absent features and is estimated in practice from a background dataset, typically a subset of the training data. We next show that the resulting empirical value function is a weighted sum of product games with one absent-factor table per background point; the neutral-factor and baseline value functions use a single such table, so they are the case $n_b = 1$ of everything that follows (Section C).

Proposition 4. Let f be a multiplicative model as in equation (7), and let $\mathcal{B} = \{\tilde{\mathbf{x}}^{(1)}, \dots, \tilde{\mathbf{x}}^{(n_b)}\}$ be a background dataset. Define $u_j^{(r)} = u_j^{(r)}(x_j)$ and $\tilde{u}_j^{(rb)} = u_j^{(r)}(\tilde{x}_j^{(b)})$. Then the empirical interventional value function is

$$\hat{v}_{\mathbf{x}}(\mathcal{S}) = \frac{1}{n_b} \sum_{r=1}^p \sum_{b=1}^{n_b} c_r \prod_{j \in \mathcal{S}} u_j^{(r)} \prod_{j \notin \mathcal{S}} \tilde{u}_j^{(rb)}. \quad (8)$$

The two sums in equation (8) do not complicate the attribution: by linearity, the Shapley values can be computed separately for each component-background pair and then summed with the same weights. Each pair contributes a product with one factor per feature, $u_j^{(r)}$ where the feature is present and $\tilde{u}_j^{(rb)}$ where it is absent, to which the integral argument of proposition 1 applies unchanged, giving a division-free formula.

Proposition 5 (Quadrature for empirical interventional values). Let $\hat{v}_{\mathbf{x}}$ be given by equation (8) and let $\{(\tau_q, \omega_q)\}_{q=1}^{m_q}$ be Gauss–Legendre nodes and weights on $[0, 1]$. Define

$$\hat{\phi}_i(\hat{v}_{\mathbf{x}}) := \frac{1}{n_b} \sum_{r=1}^p \sum_{b=1}^{n_b} c_r (u_i^{(r)} - \tilde{u}_i^{(rb)}) \sum_{q=1}^{m_q} \omega_q \prod_{j \neq i} ((1 - \tau_q) \tilde{u}_j^{(rb)} + \tau_q u_j^{(r)}). \quad (9)$$

Then $\hat{\phi}_i(\hat{v}_{\mathbf{x}}) = \phi_i(\hat{v}_{\mathbf{x}})$ for every $i \in \mathcal{D}$ whenever $m_q \geq \lceil d/2 \rceil$, irrespective of p and n_b .

Equation (9) is equation (4) with $T_{q,j} = (1 - \tau_q) \tilde{u}_j + \tau_q u_j$ in place of $(1 - \tau_q) + \tau_q u_j$, so the shared log-space evaluator of Section 3.2 applies to it verbatim, and no factor is ever divided by, which matters because absent factors vanish whenever a rule indicator is inactive at a background point. Because whole background rows are used, dependence among the absent features is preserved exactly; only the dependence between present and absent features is severed, and that by the interventional value function itself.

Time and memory complexity. Proposition 3 applies once per component-background pair, giving $O(p n_b d m_q)$ sequential work; the pairs are independent and their aggregation is one more associative

Table 1: Model classes with multiplicative structure: quantity explained, per-feature factor and number of components p . Notation: s_j are the smooth terms of a log-link GAM ($s_j(x_j) = \beta_j x_j$ for a GLM), $\pi(\mathbf{x}) = \Pr(Y=1 \mid \mathbf{x})$, h_0 the baseline hazard, $k = \prod_j k_j$ a product kernel with coefficients α_i , and rule ℓ tests the features J_ℓ with scores $q_{\ell j}$.

Model	Quantity explained	Factor $u_j^{(r)}(x_j)$	p
GLM / GAM, log link	$\mathbb{E}[Y \mid \mathbf{x}]$	$e^{s_j(x_j)}$	1
Logistic regression	odds $\pi(\mathbf{x})/(1 - \pi(\mathbf{x}))$	$e^{\beta_j x_j}$	1
Naive Bayes	posterior odds	$p(x_j \mid Y=1)/p(x_j \mid Y=0)$	1
Cox proportional hazards	relative hazard $h(t \mid \mathbf{x})/h_0(t)$	$e^{\beta_j x_j}$	1
Product-kernel predictor	$f(\mathbf{x}) = \sum_i \alpha_i k(\mathbf{x}, \mathbf{x}^{(i)})$	$k_j(x_j, x_j^{(i)})$	n
Rule / tree ensemble	$f(\mathbf{x}) = \sum_\ell c_\ell \prod_{j \in J_\ell} q_{\ell j}(x_j)$	$q_{\ell j}(x_j)$, and 1 for $j \notin J_\ell$	number of rules

reduction, so the parallel time is $O(\log d + \log m_q + \log(p n_b))$, which is $O(\log d + \log(p n_b))$ for $m_q = O(d)$. For the neutral-factor and baseline value functions $n_b = 1$, and the work is $O(p d m_q)$. Materializing all factor tables at once would require $O(p n_b d)$ memory. Since the outer sum in equation (8) is a reduction, the pairs can be processed in blocks of size B and accumulated into the running attribution vector, which reduces peak memory to $O(Bd)$ without approximation.

4.2 MODEL CLASSES WITH MULTIPLICATIVE STRUCTURE

Table 1 instantiates Definition 2 for widely used model families. In each case it suffices to read off the quantity being explained, the factors $u_j^{(r)}$ and the number of components p ; proposition 4 does the rest. Three remarks are in order.

Scale. The quantity explained is the one on which the model is multiplicative: the conditional mean of a log-link GLM or GAM, on which practitioners read counts, rates and expected costs; the odds of a classifier; the relative hazard of a Cox model. For classifiers the choice is consequential. On the log-odds scale the model is additive and attribution is immediate; on the probability scale, computing Shapley values of a logistic regression is #P-hard even under a fully factorized data distribution (Van den Broeck et al., 2022); the odds scale is the one on which the attributions are non-trivial and exactly computable, and by efficiency they decompose the difference between the odds at $\mathbf{x}$ and the average odds over the background, the counterpart of the log-odds explanations standard for classifiers (Lundberg & Lee, 2017; Lundberg et al., 2020).

Trees. In a tree ensemble each rule is a root-to-leaf path and $u_j^{(\ell)} \equiv 1$ off the path, so the degree of a leaf’s integrand is the number of distinct features tested on its path, unchanged by repeated splits on the same feature. The tree depth D therefore gives the sufficient threshold $m_q \geq \lceil D/2 \rceil$ independently of d , and rules inactive on a background point produce exactly the zero absent factors handled in Section 4.1. The tree value function, its sequential evaluation and the tree benchmarks are detailed in Section E.

Kernels. A product-kernel predictor has $p = n$ components with every feature in every component (support vector machines, kernel ridge regression and Gaussian processes with Gaussian, Laplacian or product-Matérn kernels). It is the densest case and the one where the blockwise accumulation and the log-space arithmetic of Section 3.2 matter most.

5 EXPERIMENTS

Our experiments verify two properties of QUADRASHAP in the main text: (i) the approximation error decays geometrically in m_q (Section 5.1); and (ii) on product-kernel models the method is an order of magnitude faster than the current exact baseline, PKEX-SHAPLEY (Mohammadi et al., 2025), and scales to dimensions where the baseline returns no explanation within a 300-second budget (Section 5.2). Tree ensembles are treated in Section E: on CPU against SHAP (Lundberg & Lee, 2017), FASTTREESHAP v1/v2 (Yang, 2021), LINEAR TREESHAP (Yu et al., 2022a;b) and SHAPIQ (Muschalik et al., 2024), and on GPU against Quadrature-TreeSHAP (Wettenstein et al., 2026) and GPUShap (Mitchell et al., 2022).

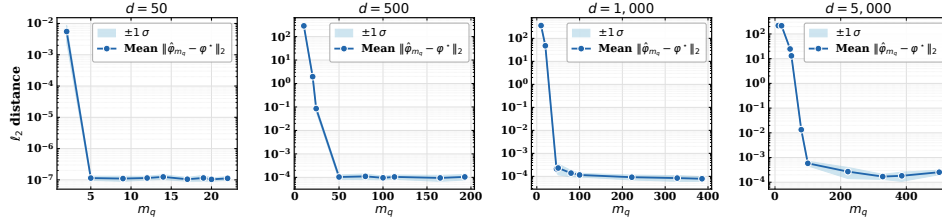


Figure 1: Mean ℓ_2 error of QUADRASHAP against the exact Shapley vector versus quadrature budget m_q (KRR with RBF kernel; bands: $\pm 1\sigma$ over ten test points; dashed line: exactness threshold). Error decays geometrically, reaching near-machine precision within a few hundred nodes at all scales.

Table 2: Kernel-explainer benchmark: runtime (seconds per instance, mean over 50 samples) on KRR with RBF kernel. *Synthetic*: varying feature dimensionality d . *TF-IDF text classification*: tuned RBF SVC at $d=5,000$ across five datasets. **T/O** denotes failure to return any explanation within the 300 seconds time limit. **Bold** marks the faster (or only completing) method.

	Synthetic (varying d)					TF-IDF text ($d=5000$)				
	50	500	1k	2k	5k	imdb	sms	sst2	RT	emotion
PKEX-SHAPLEY	0.03 ± 0.002	31.0 ± 1.6	248.3 ± 12.4	T/O	T/O	T/O	T/O	T/O	T/O	T/O
QUADRASHAP	0.11 ± 0.01	1.2 ± 0.02	2.6 ± 0.12	5.4 ± 0.21	7.6 ± 0.34	5.13 ± 0.31	2.49 ± 0.14	3.25 ± 0.18	6.21 ± 0.35	5.33 ± 0.29

5.1 APPROXIMATION QUALITY AND QUADRATURE BUDGET

We study how rapidly the approximation error decays with m_q for a product-kernel model. A Kernel Ridge Regressor (KRR) with an RBF kernel is fitted to standardized synthetic data for each $d \in \{50, 500, 1000, 5000\}$ (see Appendix F for data creation). The reference Shapley vector ϕ^* is computed once at the exactness threshold $m_q^* = \lceil d/2 \rceil$, and the mean ℓ_2 error $\|\hat{\phi} - \phi^*\|_2$ is reported as m_q varies over ten spaced values in $[1, \min(500, m_q^*)]$. Figure 1 shows the mean $\pm 1\sigma$ over ten test points. At $d=50$, the error drops three orders of magnitude from $\approx 10^{-2}$ at $m_q=2$ to $\approx 10^{-7}$ by $m_q=5$, using fewer than 20% of the nodes required for exactness. At larger scales the error is already small even at modest budgets: for $d=1,000$ it falls from ≈ 70 at $m_q=20$ to $\approx 10^{-4}$ by $m_q=50$ (exactness threshold $m_q=500$), and for $d=5,000$ a similar reduction is observed while the exactness threshold ($m_q=2,500$) lies far outside the tested range.

5.2 PRODUCT-KERNEL METHODS BENCHMARK

We benchmark QUADRASHAP against PKEX-SHAPLEY (Mohammadi et al., 2025), the current state-of-the-art exact method for product-kernel models, on synthetic and real-world data.

Synthetic data. We train a Kernel Ridge Regressor (KRR) with an RBF kernel on synthetic datasets (generated as in Section 5.1; see Appendix F) for $d \in \{50, 500, 1000, 2000, 5000\}$, and explain 50 held-out instances per setting (Table 2), with a 300-second per-instance timeout. For each d , QUADRASHAP runs with max $m_q = 400$.

PKEX-SHAPLEY is faster at small d (0.03 s vs. 0.11 s at $d=50$), but QUADRASHAP takes over at around $d \approx 150$, reaching a $25.8\times$ speedup at $d=500$ (1.2 s vs. 31.0 s) and a $95.5\times$ speedup at $d=1000$ (2.6 s vs. 248.3 s). For $d \geq 2000$, PKEX-SHAPLEY returns no explanation within the time limit, while QUADRASHAP completes in 5.4 s at $d=2000$ and 7.6 s at $d=5000$.

Real-world text classification. We further evaluate on five public text-classification datasets, emotion (Saravia et al., 2018), imdb (Maas et al., 2011), sms spam (Almeida et al., 2011), sst2 (Wang et al., 2019) and rotten tomatoes (RT) (Pang & Lee, 2005), encoded as 5,000-dimensional TF-IDF vectors (Section G), fitting and fine-tuning support vector classification (SVC) with RBF kernel on these features and explaining 50 held-out samples per dataset (Table 2). PKEX-SHAPLEY produces no explanation within 300 s on any dataset. QUADRASHAP completes on all five with $m_q = 400$, with mean per-instance runtimes from 2.49 s (sms spam) to 6.21 s (rotten tomatoes), consistent with the synthetic scaling trend.

6 CONCLUSION

We showed that Shapley values for product games reduce to a one-dimensional integral, enabling exact computation via Gauss–Legendre quadrature in $O(dm_q)$ total work and $O(\log d)$ parallel time, with geometric convergence in m_q below the exactness threshold. A log-space implementation resolves numerical instability that affects interpolation-based alternatives. Applied to tree ensembles, QUADRASHAP matches the complexity of Linear TreeSHAP while remaining numerically stable across all tested scales. Natural directions for future work include extending the quadrature framework to higher-order feature interactions (Muschalik et al., 2024), broadening the product-game formulation to additional model classes with multiplicative structure, and developing adaptive quadrature strategies that tune m_q per feature based on local convergence estimates.

REFERENCES

- Tiago A Almeida, José María G Hidalgo, and Akebo Yamakami. Contributions to the study of sms spam filtering: new collection and results. In *Proceedings of the 11th ACM symposium on Document engineering*, pp. 259–262, 2011.
- Guy E. Blelloch. Prefix sums and their applications. Technical report, Carnegie Mellon University, 1990.
- Richard P. Brent. The parallel evaluation of general arithmetic expressions. *Journal of the ACM*, 21(2):201–206, 1974.
- Siu Lun Chau, Robert Hu, Javier Gonzalez, and Dino Sejdinovic. Rkhs-shap: Shapley values for kernel methods. *Advances in neural information processing systems*, 35:13050–13063, 2022.
- Siu Lun Chau, Krikamol Muandet, and Dino Sejdinovic. Explaining the uncertain: Stochastic shapley values for gaussian process models. *Advances in Neural Information Processing Systems*, 36:50769–50795, 2023.
- Hugh Chen, Scott M. Lundberg, Gabriel G. Erion, and Su-In Lee. True to the model or true to the data? *arXiv preprint arXiv:2006.16234*, 2020.
- Philip J. Davis and Philip Rabinowitz. *Methods of Numerical Integration*. Academic Press, 2nd edition, 1984.
- Pierre Dehez. Cooperative product games. *International Game Theory Review*, 26(01):2350016, 2024.
- Fabian Fumagalli, R Teal Witter, and Christopher Musco. Polyshap: Extending kernelshap with interaction-informed polynomial regression. *arXiv preprint arXiv:2601.18608*, 2026.
- Walter Gautschi. Norm estimates for inverses of Vandermonde matrices. *Numerische Mathematik*, 23(4):337–347, 1975.
- Gene H. Golub and John H. Welsch. Calculation of Gauss quadrature rules. *Mathematics of Computation*, 23(106):221–230, 1969.
- Farzaneh Heidari, Chao Li, and Guillaume Rabusseau. Tractable shapley values and interactions via tensor networks. In *Proceedings of the 29th International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2026.
- Tom Heskes, Enzo Sijben, Ioana Bucur, and Tom Claassen. Causal shapley values: Exploiting causal knowledge to explain individual predictions of complex models. *Advances in Neural Information Processing Systems*, 33, 2020.
- Dominik Janzing, Lenon Minorics, and Patrick Blöbaum. Feature relevance quantification in explainable ai: A causal problem. *Proceedings of the 23rd International Conference on Artificial Intelligence and Statistics*, 2020.
- Zhongli Jiang, Min Zhang, and Dabao Zhang. Fast calculation of feature contributions in boosting trees. In *The 41st Conference on Uncertainty in Artificial Intelligence*, 2025.
- Weida Li, Yaoliang Yu, and Bryan Kian Hsiang Low. TreeGrad-Ranker: Feature ranking via $O(1)$ -time gradients for decision trees. In *International Conference on Learning Representations*, 2026.
- Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, 2017.
- Scott M. Lundberg, Gabriel Erion, Hugh Chen, Alex DeGrave, Jordan M. Prutkin, Bala Nair, Ronit Katz, Jonathan Himmelfarb, Nisha Bansal, and Su-In Lee. From local explanations to global understanding with explainable ai for trees. *Nature Machine Intelligence*, 2020.

- Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng, and Christopher Potts. Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pp. 142–150, Portland, Oregon, USA, June 2011. Association for Computational Linguistics. URL <http://www.aclweb.org/anthology/P11-1015>.
- Luke Merrick and Ankur Taly. The explanation game: Explaining machine learning models using shapley values. *arXiv preprint arXiv:2009.05500*, 2020.
- Rory Mitchell, Eibe Frank, and Geoffrey Holmes. Gputreeshap: massively parallel exact calculation of shap scores for tree ensembles. *PeerJ Computer Science*, 8:e880, 2022.
- Majid Mohammadi, Siu Lun Chau, and Krikamol Muandet. Computing exact shapley values in polynomial time for product-kernel methods. *arXiv preprint arXiv:2505.16516*, 2025.
- Majid Mohammadi, Krikamol Muandet, Ilaria Tiddi, Annette Ten Teije, and Siu Lun Chau. Exact shapley attributions in quadratic-time for fanova gaussian processes. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 24414–24421, 2026.
- Christoph Molnar. *Interpretable Machine Learning*. Lulu.com, 2022. URL <https://christophm.github.io/interpretable-ml-book/>.
- Maximilian Muschalik, Hubert Baniecki, Fabian Fumagalli, Patrick Kolpaczki, Barbara Hammer, and Eyke Hüllermeier. shapiq: Shapley interactions for machine learning. *Advances in Neural Information Processing Systems*, 37:130324–130357, 2024.
- Guillermo Owen. Multilinear extensions of games. *Management Science*, 18(5-part-2):64–79, 1972.
- Bo Pang and Lillian Lee. Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. In *Proceedings of the ACL*, 2005.
- Elvis Saravia, Hsien-Chi Toby Liu, Yen-Hao Huang, Junlin Wu, and Yi-Shin Chen. CARER: Contextualized affect representations for emotion recognition. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pp. 3687–3697, Brussels, Belgium, October–November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1404. URL <https://www.aclweb.org/anthology/D18-1404>.
- Lloyd S. Shapley. A value for n-person games. In *Contributions to the Theory of Games*. Princeton University Press, 1953.
- Mukund Sundararajan and Amir Najmi. The many shapley values for model explanation. *Proceedings of the 37th International Conference on Machine Learning*, 2020.
- Lloyd N. Trefethen. Is Gauss quadrature better than Clenshaw–Curtis? *SIAM Review*, 50(1):67–87, 2008.
- Lloyd N. Trefethen. *Approximation Theory and Approximation Practice*. SIAM, 2013.
- Guy Van den Broeck, Anton Lykov, Maximilian Schleich, and Dan Suciu. On the tractability of SHAP explanations. *Journal of Artificial Intelligence Research*, 74, 2022. Preliminary version appeared at AAAI 2021.
- Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. 2019. In the Proceedings of ICLR.
- Ron Wettenstein, Rory Mitchell, and Peng Yu. Quadrature-TreeSHAP: Depth-independent TreeSHAP and Shapley interactions. *arXiv preprint arXiv:2605.04497*, 2026.
- Jilei Yang. Fast treeshap: Accelerating shap value computation for trees. *arXiv preprint arXiv:2109.09847*, 2021.
- Peng Yu, Albert Bifet, Jesse Read, Chao Xu, et al. Linear treeshap. In *Advances in Neural Information Processing Systems*, 2022a.

594 Peng Yu, Albert Bifet, Jesse Read, Chao Xu, et al. Linear treeshap: reference implementation.
595 https://github.com/yupbank/linear_tree_shap, 2022b. Accessed: 2026-04-12,
596 commit: 9deb5568.
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647

A RELATION TO QUADRATURE-BASED TREE EXPLAINERS

Two recent methods evaluate Shapley values of decision trees by Gauss–Legendre quadrature. We summarize both, state what the three approaches share, and then list where they differ. Table 3 condenses the comparison; the theoretical and empirical comparison specific to trees, including the CPU and GPU benchmarks, is given in Section E.

TreeGrad-Shap. Li et al. (2026) start from Owen’s multilinear extension $\bar{f}_x$ of the tree game (Owen, 1972) and the identity $\phi_i = \int_0^1 \partial_i \bar{f}_x(t\mathbf{1}) dt$. Along the diagonal $t\mathbf{1}$ the gradient is a polynomial of degree at most $D - 1$ for a tree of depth D , so a Gauss–Legendre rule with $\lceil D/2 \rceil$ nodes evaluates the integral exactly, in $O(LD)$ time for a tree with L leaves; the per-leaf polynomial is accumulated along the root-to-leaf path as a product of factors of the form $(1 - t) + t\gamma_e$, with the factor of an ancestor divided out when a feature is split on repeatedly. The method is presented as a by-product of the paper’s main contribution, feature ranking by gradient optimization, and also yields weighted Banzhaf and Beta-Shapley values. It is presented for single decision trees under the path-dependent (cover-based) value function of TreeSHAP (Lundberg et al., 2020); ensembles follow by linearity. Numerical stability relative to Linear TreeSHAP is a headline claim, supported by experiments; the authors note that the path products can underflow for deep trees and leave this unaddressed.

Quadrature-TreeSHAP. Concurrently with the present work, Wettstein et al. (2026) reformulate path-dependent TreeSHAP as integrals over $p \in [0, 1]$ of “weighted Banzhaf polynomials” of the form $\prod_{j \in S} (q_j - 1) \prod_{j \notin S} ((1 - p) + p q_j)$ per leaf, which for $|S| = 1$ is the integrand of proposition 1 restricted to the tree game, and for $|S| > 1$ yields Shapley interaction indices of any order. The integrals are evaluated by Gauss–Legendre quadrature. The paper’s central observation is empirical: although exactness requires $\lceil (\eta - |S| + 1)/2 \rceil$ nodes, where η is the largest number of distinct features on a root-to-leaf path, a fixed budget of eight nodes reaches the numerical noise floor of the single-precision output in all their experiments, so runtime becomes practically independent of tree depth. The C++ CPU implementation has been merged into XGBoost as its TreeSHAP backend; a GPU implementation is reported separately. The method is confined to tree ensembles and the path-dependent value function.

What the three approaches share. The integral representation is the same classical identity in all three cases. For a product game the multilinear extension is $\bar{v}(\mathbf{z}) = \prod_j ((1 - z_j) + z_j u_j)$, so Owen’s identity yields equation (3) directly; our derivation via the Beta representation of the Shapley weights arrives at the same formula. All three methods then apply Gauss–Legendre quadrature to a polynomial integrand and obtain exactness once the node count exceeds half the degree. For path-dependent trees in particular, the three methods evaluate the same per-leaf polynomial at the same nodes, at the same $O(L m_q)$ cost per tree; the tree instantiation of QUADRASHAP should be read in that light.

Where they differ. The differences lie in the object to which the identity is applied and in the regime the resulting algorithm is built for.

Level of generality. Both tree methods apply the identity to the tree game and are tied to the path-dependent value function. QUADRASHAP applies it to the product game as an abstract object and shows in Section 4 that the empirical interventional value function of any model with a sum-of-products representation is an exact weighted sum of such games. This is what carries the rule to product-kernel machines, log-link GLMs and GAMs, Cox models, odds-scale classifiers and rule ensembles, none of which is reachable from the tree formulation.

Polynomial degree. For a tree the degree is bounded by the number of distinct features on a path, rarely more than a few tens. For the product games of Section 4 it is d_{eff} , the number of features on which the explained instance and the background differ, which for kernels and GLMs is essentially the number of features and routinely reaches thousands. Exactness then costs $O(d^2)$, so the approximation regime with provable convergence rate is a necessity rather than an optimization.

Approximation guarantee. TreeGrad-Shap uses the exact node count only. Quadrature-TreeSHAP fixes eight nodes on empirical grounds relative to a single-precision noise floor of about 10^{-7} ; its

only theoretical statement about the node count is the exactness threshold. Proposition 2 gives a geometric error bound below the threshold.

Dynamic range. A product of thousands of factors can leave the range of double precision. TreeGrad-Shap acknowledges underflow for deep trees; neither tree method handles it. QUADRASHAP evaluates all products in log-space with explicit sign tracking (Section 3.2), which is what allows the high-dimensional benchmarks of Section 5.

Parallel structure. QUADRASHAP states the computation as associative reductions and obtains $O(\log d)$ parallel time (Proposition 3). Quadrature-TreeSHAP provides SIMD-vectorized CPU code and a GPU implementation but does not analyze parallel time; TreeGrad-Shap does not discuss parallelism.

Interactions. Quadrature-TreeSHAP computes Shapley interaction indices of any order for trees. The identity extends to product games in the same way (Section 4.1), but we do not implement or evaluate interaction indices in this paper.

In summary, on path-dependent trees the first-order specializations of the three methods evaluate the same leaf-wise quadrature integrand. Section E benchmarks QUADRASHAP on CPU against the established non-quadrature tree explainers and on GPU against the XGBoost implementation of Quadrature-TreeSHAP and against GPUShap, with the CPU implementation of TreeGrad-Shap compared separately; as discussed there, the GPU differences reflect implementation and call organization rather than a different algorithm. The contribution of QUADRASHAP is the model-independent formulation, the model classes it brings within reach, and the approximation, range, and parallelism results needed once the polynomial degree is the number of features rather than the depth of a tree.

Table 3: Quadrature-based Shapley methods compared. D is tree depth, η the largest number of distinct features on a root-to-leaf path, and d_{eff} the number of features on which present and absent factors differ.

	TREEGRAD-SHAP (Li et al., 2026)	Quadrature-TreeSHAP (Wettenstein et al., 2026)	QUADRASHAP (this paper)
Object of the identity	tree game (multilinear extension)	tree game (weighted Banzhaf polynomial)	product game, stated independently of any model
Value functions	path-dependent tree	path-dependent tree	empirical interventional for sum-of-products models; path-dependent tree; kernel-specific
Model scope	decision trees (ensembles by linearity)	tree ensembles	GLM/GAM (log link), Cox, odds-scale classifiers, product kernels, rule ensembles, trees
Polynomial degree	$\leq$ tree depth D	$\leq$ distinct path features η	d_{eff} , up to the number of features
Exact node count	$\lceil D/2 \rceil$	$\lceil (\eta - \mathcal{S} + 1)/2 \rceil$	$\lceil d_{\text{eff}}/2 \rceil$
Sub-threshold regime	not considered	fixed 8 nodes, empirical (single-precision floor)	geometric bound (Proposition 2); budget set empirically
Interaction indices	no	any order	identity stated; not implemented
Dynamic range	underflow noted, unhandled	not addressed	log-space with sign tracking
Parallelism	not discussed	SIMD CPU, GPU implementation	$O(\log d)$ parallel time via associative reductions

B A DIVISION-FREE VARIANT VIA PREFIX-SUFFIX SCANS

The evaluator of Section 3.2 forms the full product P_q once and recovers every leave-one-out product by a division, which is what forces the separate treatment of vanishing factors. The leave-one-out products can instead be assembled directly from an *exclusive prefix* and an *exclusive suffix* product,

Algorithm 1 Division-free evaluation of equation (4) by prefix-suffix scans

Require: factors $u_1, \dots, u_d$; Gauss-Legendre nodes and weights $\{(\tau_q, \omega_q)\}_{q=1}^{m_q}$

Ensure: $\hat{\phi}_1, \dots, \hat{\phi}_d$

```

1:  $\hat{\phi}_i \leftarrow 0$  for all  $i$ 
2: for  $q = 1$  to  $m_q$  do                                     ▷ independent across  $q$ 
3:    $T_j \leftarrow (1 - \tau_q) + \tau_q u_j$  for all  $j$                  ▷ parallel over  $j$ 
4:    $a_j \leftarrow (\log |T_j|, \text{sgn}(T_j), 0)$  if  $T_j \neq 0$ , else  $a_j \leftarrow (0, 1, 1)$ , for all  $j$ 
5:    $A^< \leftarrow \text{EXCLUSIVEPREFIXSCAN}((a_j)_{j=1}^d; \oplus)$          ▷  $O(d)$  work,  $O(\log d)$  parallel time
6:    $A^> \leftarrow \text{EXCLUSIVESUFFIXSCAN}((a_j)_{j=1}^d; \oplus)$ 
7:   for  $i = 1$  to  $d$  do                                         ▷ parallel over  $i$ 
8:      $(\ell, s, z) \leftarrow A_i^< \oplus A_i^>$ 
9:      $\hat{\phi}_i \leftarrow \hat{\phi}_i + \omega_q (u_i - 1) [z = 0] s e^\ell$ 
10:  end for
11: end for

```

$\prod_{j \neq i} T_{q,j} = (\prod_{j < i} T_{q,j}) (\prod_{j > i} T_{q,j})$, which requires neither a division nor a special case. To carry this out in log-space with zeros allowed, attach to each factor the triple

$$a_{q,j} := \begin{cases} (\log |T_{q,j}|, \text{sgn}(T_{q,j}), 0), & T_{q,j} \neq 0, \\ (0, 1, 1), & T_{q,j} = 0, \end{cases} \quad (\ell_1, s_1, z_1) \oplus (\ell_2, s_2, z_2) := (\ell_1 + \ell_2, s_1 s_2, z_1 + z_2),$$

so that a triple records the log-magnitude and sign of the product of its non-zero factors together with the number of zero factors it contains. The operation $\oplus$ is associative and commutative with identity $(0, 1, 0)$, and a triple (ℓ, s, z) decodes to the product $[z = 0] s e^\ell$. Writing $A_{q,i}^< := a_{q,1} \oplus \dots \oplus a_{q,i-1}$ and $A_{q,i}^> := a_{q,i+1} \oplus \dots \oplus a_{q,d}$ for the exclusive prefix and suffix reductions, the estimator equation (4) becomes

$$\hat{\phi}_i(v) = (u_i - 1) \sum_{q=1}^{m_q} \omega_q \text{dec}(A_{q,i}^< \oplus A_{q,i}^>), \quad \text{dec}(\ell, s, z) := [z = 0] s e^\ell. \quad (10)$$

Algorithm 1 states the procedure. All $A_{q,i}^<$ are produced by one exclusive prefix scan of $(a_{q,j})_j$ under $\oplus$, and all $A_{q,i}^>$ by one exclusive suffix scan, each in $O(d)$ work and $O(\log d)$ parallel time (Blelloch, 1990); the combine and decode steps are $O(1)$ per pair (i, q) . Proposition 3 therefore holds verbatim for this variant. Sequentially it makes three passes over the d factors per node instead of two, a constant-factor overhead, and it performs no divisions. The same construction applies unchanged to the factors $(1 - \tau_q) \tilde{u}_j + \tau_q u_j$ of Section 4.

Comparison with the shared-product evaluator. The two evaluators compute the same quantity whenever no factor vanishes, and their asymptotic work and parallel time coincide. They differ in three respects. The shared-product form of Section 3.2 uses the scan primitive only as a reduction and needs one division per pair (i, q) ; the prefix-suffix form uses two genuine scans and no division. The shared-product form must detect $\text{sgn}(P_q) = 0$ and recompute the affected leave-one-out products separately, whereas the prefix-suffix form treats zero factors through the counter z with no change of control flow, which matters when many components have vanishing absent factors. Finally, per node and feature the shared-product form exponentiates a difference of two accumulated logarithms, while the prefix-suffix form exponentiates a single accumulated sum; the rounding behavior of the two is the same up to that last operation.

C VALUE FUNCTIONS FOR MULTIPLICATIVE MODELS

This appendix collects the value functions to which proposition 5 applies and makes precise the claim of Section 4.1 that the neutral-factor and baseline value functions are simpler cases of the empirical interventional one: they fill in a single table of absent factors, whereas the interventional value function averages over n_b such tables, one per background point.

A common form. Let f be a multiplicative model as in equation (7). Every value function below can be written as

$$v_{\mathbf{x}}(\mathcal{S}) = \sum_{r=1}^p \sum_{b=1}^{n_b} w_{rb} \prod_{j \in \mathcal{S}} u_j^{(r)} \prod_{j \notin \mathcal{S}} \tilde{u}_j^{(rb)}, \quad u_j^{(r)} = u_j^{(r)}(x_j), \quad (11)$$

for weights $w_{rb} \in \mathbb{R}$ and absent factors $\tilde{u}_j^{(rb)} \in \mathbb{R}$ that do not depend on $\mathcal{S}$. For each pair (r, b) the coalition value is a product with one factor per feature, $u_j^{(r)}$ if the feature is present and $\tilde{u}_j^{(rb)}$ if it is absent. The proof of proposition 5 in Section D uses only this structure: it treats one pair at a time, never divides by an absent factor, and combines the pairs by linearity of the Shapley value. It therefore gives, for any choice of w_{rb} and $\tilde{u}_j^{(rb)}$,

$$\phi_i(v_{\mathbf{x}}) = \sum_{r=1}^p \sum_{b=1}^{n_b} w_{rb} (u_i^{(r)} - \tilde{u}_i^{(rb)}) \int_0^1 \prod_{j \neq i} ((1-t) \tilde{u}_j^{(rb)} + t u_j^{(r)}) dt, \quad (12)$$

and the m_q -point Gauss–Legendre rule evaluates every integral exactly whenever $m_q \geq \lceil d/2 \rceil$, since each integrand is a polynomial of degree at most $d-1$ in t . The value functions differ only in how the tables $\tilde{u}^{(rb)}$ and the weights w_{rb} are filled in, and the cost of equation (12) is $O(p n_b d m_q)$, proportional to the number of tables.

Empirical interventional value function. Let $\mathcal{B} = \{\tilde{\mathbf{x}}^{(1)}, \dots, \tilde{\mathbf{x}}^{(n_b)}\}$ be a background dataset. The interventional value function $v_{\mathbf{x}}^{\text{int}}(\mathcal{S}) = \mathbb{E}_{X_{\mathcal{D} \setminus \mathcal{S}}} [f(\mathbf{x}_{\mathcal{S}}, X_{\mathcal{D} \setminus \mathcal{S}})]$ of Section 2 is estimated by replacing the expectation with the average over $\mathcal{B}$,

$$\hat{v}_{\mathbf{x}}(\mathcal{S}) = \frac{1}{n_b} \sum_{b=1}^{n_b} f(\mathbf{x}_{\mathcal{S}}, \tilde{\mathbf{x}}_{\mathcal{D} \setminus \mathcal{S}}^{(b)}) = \frac{1}{n_b} \sum_{b=1}^{n_b} \sum_{r=1}^p c_r \prod_{j \in \mathcal{S}} u_j^{(r)}(x_j) \prod_{j \notin \mathcal{S}} u_j^{(r)}(\tilde{x}_j^{(b)}), \quad (13)$$

where the second equality substitutes equation (7) and uses that each factor depends on one coordinate only, so that the coordinates of the mixed point $(\mathbf{x}_{\mathcal{S}}, \tilde{\mathbf{x}}_{\mathcal{D} \setminus \mathcal{S}}^{(b)})$ separate. This is equation (11) with n_b tables, $\tilde{u}_j^{(rb)} = u_j^{(r)}(\tilde{x}_j^{(b)})$ and $w_{rb} = c_r/n_b$, which is proposition 4. Because whole background rows are used, the joint distribution of the absent features enters exactly; $\hat{v}_{\mathbf{x}}(\emptyset)$ is the mean prediction over $\mathcal{B}$, so by efficiency the attributions sum to $f(\mathbf{x}) - \frac{1}{n_b} \sum_b f(\tilde{\mathbf{x}}^{(b)})$.

Baseline value function. Fix a single reference point $\mathbf{x}^b$ and set $v_{\mathbf{x}}(\mathcal{S}) = f(\mathbf{x}_{\mathcal{S}}, \mathbf{x}_{\mathcal{D} \setminus \mathcal{S}}^b)$. Repeating the computation in equation (13) with $\mathcal{B} = \{\mathbf{x}^b\}$ gives

$$v_{\mathbf{x}}(\mathcal{S}) = \sum_{r=1}^p c_r \prod_{j \in \mathcal{S}} u_j^{(r)}(x_j) \prod_{j \notin \mathcal{S}} u_j^{(r)}(x_j^b), \quad (14)$$

which is equation (11) with one table, $\tilde{u}_j^{(r)} = u_j^{(r)}(x_j^b)$ and $w_r = c_r$. The empirical interventional value function is the average of n_b baseline value functions, one per background point, and by linearity its Shapley values are the averages of the corresponding baseline Shapley values; the baseline value function is thus exactly the case $n_b = 1$ of proposition 5, at cost $O(p d m_q)$. Its attributions sum to $f(\mathbf{x}) - f(\mathbf{x}^b)$.

Neutral factor. The neutral factor sets $\tilde{u}_j^{(r)} \equiv 1$ for every component and every feature, so that an absent feature is simply dropped from each product:

$$v_{\mathbf{x}}(\mathcal{S}) = \sum_{r=1}^p c_r \prod_{j \in \mathcal{S}} u_j^{(r)}, \quad v_{\mathbf{x}}(\emptyset) = \sum_{r=1}^p c_r. \quad (15)$$

This is a weighted sum of product games in the sense of Definition 1, with the convention $v(\emptyset) = 1$ for each of them, and it is the value function for which Section 3 was developed: with $\tilde{u}_j = 1$, equation (12) reduces to $c_r(u_i^{(r)} - 1) \int_0^1 \prod_{j \neq i} ((1-t) + t u_j^{(r)}) dt$, the integral identity of proposition 1, and equation (4) is its quadrature. For a product-kernel predictor $f(\mathbf{x}) = \sum_{i=1}^n \alpha_i \prod_j k_j(x_j, x_j^{(i)})$ the neutral factor gives $v_{\mathbf{x}}(\mathcal{S}) = \sum_i \alpha_i \prod_{j \in \mathcal{S}} k_j(x_j, x_j^{(i)})$, that is, the predictor evaluated with the

kernel restricted to the present features, which is the value function used by Mohammadi et al. (2025); its attributions sum to $f(\mathbf{x}) - \sum_i \alpha_i$. Unlike the baseline value function, the neutral factor need not correspond to any reference point, since there need not be an $\mathbf{x}^b$ with $u_j^{(r)}(x_j^b) = 1$ for all r simultaneously. Algorithmically, however, it is again the case $n_b = 1$ of proposition 5, with the single absent-factor table filled with ones, at cost $O(p d m_q)$.

Mean absent factor. A fourth option sits between the baseline and the empirical interventional value functions. If the background distribution factorizes across features, then $\mathbb{E} \prod_{j \notin \mathcal{S}} u_j^{(r)}(X_j) = \prod_{j \notin \mathcal{S}} \mathbb{E} u_j^{(r)}(X_j)$, so the interventional value function is itself of the form equation (11) with a single table, $\tilde{u}_j^{(r)} = \mathbb{E} u_j^{(r)}(X_j)$, estimated by the per-feature averages $\frac{1}{n_b} \sum_b u_j^{(r)}(\tilde{x}_j^{(b)})$. This costs $O(p d m_q)$ after an $O(p n_b d)$ pass over the background data, against $O(p n_b d m_q)$ for equation (13), but it multiplies averages where equation (13) averages products: the two coincide only when the absent features are independent, and the empirical interventional value function is the one that preserves their dependence.

Table 4: Value functions for a multiplicative model as instances of equation (11). Each is handled by proposition 5 with the stated number of absent-factor tables, at $O(p d m_q)$ work per table; $\bar{u}_j^{(r)} = \frac{1}{n_b} \sum_b u_j^{(r)}(\tilde{x}_j^{(b)})$ and $\bar{f}_B = \frac{1}{n_b} \sum_b f(\tilde{\mathbf{x}}^{(b)})$.

Value function	Absent factor $\tilde{u}_j^{(r,b)}$	Tables	$\sum_i \phi_i$
Neutral factor (Mohammadi et al., 2025)	1	1	$f(\mathbf{x}) - \sum_r c_r$
Baseline $\mathbf{x}^b$	$u_j^{(r)}(x_j^b)$	1	$f(\mathbf{x}) - f(\mathbf{x}^b)$
Mean absent factor	$\tilde{u}_j^{(r)}$	1	$f(\mathbf{x}) - \sum_r c_r \prod_j \tilde{u}_j^{(r)}$
Empirical interventional	$u_j^{(r)}(\tilde{x}_j^{(b)})$	n_b	$f(\mathbf{x}) - \bar{f}_B$

D PROOFS

PROOF OF PROPOSITION 1

By definition, the Shapley value of player $i \in \mathcal{D}$ is

$$\phi_i(v) = \sum_{\mathcal{S} \subseteq \mathcal{D} \setminus \{i\}} \frac{|\mathcal{S}|! (d - |\mathcal{S}| - 1)!}{d!} (v(\mathcal{S} \cup \{i\}) - v(\mathcal{S})).$$

For a product game, the marginal contribution simplifies to

$$v(\mathcal{S} \cup \{i\}) - v(\mathcal{S}) = \left(\prod_{j \in \mathcal{S}} u_j \right) (u_i - 1).$$

Substituting this expression yields

$$\phi_i(v) = (u_i - 1) \sum_{\mathcal{S} \subseteq \mathcal{D} \setminus \{i\}} \mu_{|\mathcal{S}|} \prod_{j \in \mathcal{S}} u_j, \quad \mu_q := \frac{q! (d - q - 1)!}{d!}.$$

We now express the coefficients μ_q as integrals. For integers $q \in \{0, \dots, d - 1\}$, consider the integral

$$\int_0^1 t^q (1 - t)^{d - q - 1} dt.$$

By repeated integration by parts (or by the definition of the Beta function), this integral evaluates to

$$\int_0^1 t^q (1 - t)^{d - q - 1} dt = \frac{q! (d - q - 1)!}{d!} = \mu_q.$$

Thus, each Shapley weight μ_q admits the representation

$$\mu_q = \int_0^1 t^q (1 - t)^{d - q - 1} dt.$$

Substituting this identity into the Shapley expression and interchanging summation and integration give

$$\phi_i(v) = (u_i - 1) \int_0^1 \sum_{S \subseteq \mathcal{D} \setminus \{i\}} t^{|S|} (1-t)^{d-|S|-1} \prod_{j \in S} u_j dt.$$

To simplify the inner sum, observe that for each feature $j \neq i$, the term inside the summation contributes either a factor $(1-t)$ if $j \notin S$ or a factor $t u_j$ if $j \in S$. Summing over all subsets $S \subseteq \mathcal{D} \setminus \{i\}$ therefore corresponds to expanding the product

$$\prod_{j \neq i} ((1-t) + t u_j).$$

Since $|\mathcal{D} \setminus \{i\}| = d - 1$, this expansion yields

$$\sum_{S \subseteq \mathcal{D} \setminus \{i\}} t^{|S|} (1-t)^{d-|S|-1} \prod_{j \in S} u_j = \prod_{j \neq i} ((1-t) + t u_j).$$

Substituting back into the integral completes the proof:

$$\phi_i(v) = (u_i - 1) \int_0^1 \prod_{j \neq i} ((1-t) + t u_j) dt.$$

PROOF OF PROPOSITION 2

Part (i): Exactness. The integrand $g_i(t) := \prod_{j \neq i} ((1-t) + t u_j)$ is a polynomial of degree at most $d - 1$ in t . An m_q -point GL rule on $[0, 1]$ integrates exactly every polynomial of degree at most $2m_q - 1$ (Davis & Rabinowitz, 1984). For $m_q \geq \lceil d/2 \rceil$ we have $2m_q - 1 \geq 2\lceil d/2 \rceil - 1 \geq d - 1$, so the quadrature sum reproduces the exact integral:

$$\sum_{q=1}^{m_q} \omega_q g_i(\tau_q) = \int_0^1 g_i(t) dt.$$

Multiplying both sides by $(u_i - 1)$ and applying Proposition 1 gives $\widehat{\phi}_i(v) = \phi_i(v)$.

Part (ii): Approximation. Let

$$g_i(t) := \prod_{j \neq i} ((1-t) + t u_j).$$

Given Proposition 1, one can write:

$$\begin{aligned} |\phi_i(v) - \widehat{\phi}_i(v)| &= \left| (u_i - 1) \int_0^1 g_i(t) dt - (u_i - 1) \sum_{q=1}^{m_q} \omega_q g_i(\tau_q) \right| \\ &= |u_i - 1| \left| \int_0^1 g_i(t) dt - \sum_{q=1}^{m_q} \omega_q g_i(\tau_q) \right|. \end{aligned} \quad (16)$$

Now g_i is a polynomial in t of degree at most $d - 1$, hence it is entire and in particular analytic on and inside every Bernstein ellipse E_ρ , for any $\rho > 1$. Standard Gauss–Legendre quadrature error bounds for analytic functions on $[-1, 1]$ (Trefethen, 2013, Thm. 19.3), transferred to $[0, 1]$ via the affine change of variables $x = 2t - 1$, imply

$$\left| \int_0^1 g_i(t) dt - \sum_{q=1}^{m_q} \omega_q g_i(\tau_q) \right| \leq \frac{4 \max_{z \in E_\rho} |g_i(z)|}{\rho^{2m_q} (\rho^2 - 1)}.$$

Substituting this bound into the previous display yields

$$|\phi_i(v) - \widehat{\phi}_i(v)| \leq |u_i - 1| \frac{4 \max_{z \in E_\rho} |g_i(z)|}{\rho^{2m_q} (\rho^2 - 1)}.$$

Hence, for any fixed $\rho > 1$,

$$|\phi_i(v) - \hat{\phi}_i(v)| \leq C_\rho M_\rho |u_i - 1| \rho^{-2m_q},$$

where $M_\rho := \max_{z \in E_\rho} |g_i(z)|$ and $C_\rho := 4/(\rho^2 - 1)$ depends only on ρ . This proves the claimed geometric decay of the approximation error. $\square$

PROOF OF PROPOSITION 3

Sequential. Computing, for each $q \in \{1, \dots, m_q\}$, the factors $T_{q,j} = (1 - \tau_q) + \tau_q u_j$ and the associated quantities $\log |T_{q,j}|$ and $\text{sgn}(T_{q,j})$ costs $O(dm_q)$ arithmetic operations in total. Accumulating $\log |P_q| = \sum_j \log |T_{q,j}|$ and $\text{sgn}(P_q) = \prod_j \text{sgn}(T_{q,j})$ is a length- d reduction for each of the m_q nodes, contributing $O(dm_q)$ additional work. Evaluating equation (6) then requires, for each $i \in \{1, \dots, d\}$, a weighted sum of m_q terms, each obtained from $\log |P_q| - \log |T_{q,i}|$ in constant time, for a further $O(dm_q)$ operations. The total is therefore $O(dm_q)$.

Parallel time. Given an associative operator $\oplus$ and a length- d sequence, a tree reduction, the reduction case of the associative scan, evaluates $a_1 \oplus \dots \oplus a_d$ in parallel time $O(\log d)$ using $O(d)$ processors (Blelloch, 1990). Both $\log |P_q|$ (under addition) and $\text{sgn}(P_q)$ (under multiplication) are such reductions over the feature axis of length d , and the m_q nodes are independent, so the shared-product phase completes in parallel time $O(\log d)$. For each i , evaluating equation (6) requires (i) subtracting $\log |T_{q,i}|$ from $\log |P_q|$ pointwise (parallel time $O(1)$), and (ii) a weighted sum over q , which is a length- m_q tree reduction with parallel time $O(\log m_q)$. Since the d Shapley values are independent given $\{(\log |P_q|, \text{sgn}(P_q))\}_{q=1}^{m_q}$, they are evaluated in parallel across i without increasing parallel time. The overall parallel time is therefore $O(\log d + \log m_q) = O(\log d)$. The required number of processors is the ratio of total work to parallel time (Brent, 1974), giving $O(dm_q/(\log d + \log m_q))$ processors. The exact regime $m_q = \lceil d/2 \rceil$ is a special case. $\square$

PROOF OF PROPOSITION 5

By linearity of the Shapley value it suffices to treat one component-background pair, that is the game $w(\mathcal{S}) = \prod_{j \in \mathcal{S}} u_j \prod_{j \notin \mathcal{S}} \tilde{u}_j$ for arbitrary $u_j, \tilde{u}_j \in \mathbb{R}$, where the superscripts (r) and (rb) are omitted. Its marginal contributions are

$$w(\mathcal{S} \cup \{i\}) - w(\mathcal{S}) = (u_i - \tilde{u}_i) \prod_{j \in \mathcal{S}} u_j \prod_{j \notin \mathcal{S} \cup \{i\}} \tilde{u}_j.$$

Substituting the Beta representation $\mu(s) = \int_0^1 t^s (1-t)^{d-s-1} dt$ of the Shapley weights into equation (1) and exchanging the finite sum with the integral,

$$\phi_i(w) = (u_i - \tilde{u}_i) \int_0^1 \sum_{\mathcal{S} \subseteq \mathcal{D} \setminus \{i\}} \prod_{j \in \mathcal{S}} t u_j \prod_{j \in \mathcal{D} \setminus \mathcal{S} \setminus \{i\}} (1-t) \tilde{u}_j dt = (u_i - \tilde{u}_i) \int_0^1 \prod_{j \neq i} ((1-t) \tilde{u}_j + t u_j) dt,$$

since the subset sum is precisely the expansion of the product over the $d-1$ features $j \neq i$. No factor is inverted, so the identity holds also when some $\tilde{u}_j$ vanish. The integrand is a polynomial of degree at most $d-1$ in t , so the m_q -point Gauss-Legendre rule evaluates the integral exactly as soon as $2m_q - 1 \geq d-1$, that is $m_q \geq \lceil d/2 \rceil$ (Davis & Rabinowitz, 1984). Summing over the $p n_b$ pairs with the weights c_r/n_b gives equation (9). $\square$

E TREE-BASED MODELS

This appendix collects the material on tree ensembles: the path-dependent value function and its leaf-wise product structure (Section E.1), the sequential evaluation and its relation to the other quadrature-based tree explainers (Section E.2), the CPU benchmark against the established tree explainers (Section E.3), and the GPU benchmark against Quadrature-TreeSHAP and GPUShAP (Section E.4).

E.1 PATH-DEPENDENT VALUE FUNCTION

Consider a decision tree with leaf set $\mathcal{L}$ and edge set $\mathcal{E}$. Each leaf $\ell \in \mathcal{L}$ stores a prediction ν_ℓ , and $\mathcal{E}_\ell \subseteq \mathcal{E}$ denotes the edges on the root-to- ℓ path. In the path-dependent (cover-based) value function of TreeSHAP (Lundberg et al., 2020), a coalition $\mathcal{S}$ is treated as observed by resolving the splits on features in $\mathcal{S}$ deterministically for $\mathbf{x}$, while splits on features outside $\mathcal{S}$ are averaged with the branch probabilities. For an edge $e \in \mathcal{E}$ let $p_e \in (0, 1)$ be the fraction of training samples taking that edge and $\kappa(e) \in \mathcal{D}$ the feature of the split, and let $c_e(\mathbf{x}) = 1/p_e$ if $\mathbf{x}$ follows e and $c_e(\mathbf{x}) = 0$ otherwise. Grouping the edges of a path by feature defines the path factor

$$q_{j,\ell}(\mathbf{x}) = \prod_{e \in \mathcal{E}_\ell: \kappa(e)=j} c_e(\mathbf{x}),$$

with the empty product equal to one, so that $q_{j,\ell}(\mathbf{x}) = 1$ for every feature not tested on the path. The value function is then

$$v_{\mathbf{x}}(\mathcal{S}) = \sum_{\ell \in \mathcal{L}} \nu_\ell \left(\prod_{e \in \mathcal{E}_\ell} p_e \right) \prod_{j \in \mathcal{S}} q_{j,\ell}(\mathbf{x}), \quad (17)$$

a weighted sum of product games in the sense of Definition 1, one per leaf, with factors $u_j = q_{j,\ell}(\mathbf{x})$ and weights $\nu_\ell \prod_{e \in \mathcal{E}_\ell} p_e$; since the branch probabilities are positive, the normalization of Definition 1 is always available here. Only the features tested on the path to ℓ enter the product, so the degree of the leaf’s integrand is the number η_ℓ of distinct features on that path, at most the path length and unchanged by repeated splits on the same feature; we write $\eta = \max_\ell \eta_\ell$. Ensembles follow by linearity.

Applying Proposition 2 leaf by leaf, with $a_r(q) = 1 - \tau_r + \tau_r q$, $s_r(q) = (q - 1)/a_r(q)$ and $G_\ell(r) = \nu_\ell (\prod_{e \in \mathcal{E}_\ell} p_e) \prod_{j \in \mathcal{D}} a_r(q_{j,\ell}(\mathbf{x}))$,

$$\hat{\phi}_i = \sum_{r=1}^{m_q} \omega_r \sum_{\ell \in \mathcal{L}} G_\ell(r) s_r(q_{i,\ell}(\mathbf{x})), \quad (18)$$

which is exact for $m_q \geq \lceil \eta/2 \rceil$ irrespective of the number of features d . The factor s_r divides the contribution of feature i out of the shared product $G_\ell(r)$, exactly as in Section 3.2; the division is safe because $a_r(q) \geq 1 - \tau_r > 0$ for $q \geq 0$.

E.2 SEQUENTIAL EVALUATION AND RELATION TO OTHER QUADRATURE-BASED TREE EXPLAINERS

Evaluating equation (18) leaf by leaf costs $O(m_q |\mathcal{L}| \eta)$, or $O(|\mathcal{L}| \eta^2)$ in the exact regime, the cost of the original TreeSHAP. The redundancy across leaves is removed by the telescoping reorganization of Linear TreeSHAP (Yu et al., 2022a), which carries over to the quadrature representation unchanged. Traversing an edge e with $\kappa(e) = i$ changes the state of feature i from q_e^- to $q_e^+ = q_e^- c_e(\mathbf{x})$, and the leaf-wise sum in equation (18) can be reindexed over edges: with $H_e(r) = \sum_{\ell \text{ below } e} G_\ell(r)$ and $\Delta_e(r) = s_r(q_e^+) - s_r(q_e^-)$,

$$\hat{\phi}_i = \sum_{r=1}^{m_q} \omega_r \sum_{e \in \mathcal{E}: \kappa(e)=i} H_e(r) \Delta_e(r). \quad (19)$$

A single depth-first traversal maintains the current path state, aggregates the leaf weights $H_e(r)$ upward and applies the increment $\Delta_e(r)$ once per edge, so every edge costs $O(m_q)$ and the whole tree $O(m_q |\mathcal{L}|)$, that is $O(|\mathcal{L}| \eta)$ in the exact regime; the leaf-feature table $q_{i,\ell}$ is never materialized. This is the complexity of Linear TreeSHAP without its interpolation step, which is what removes the instability documented below.

On a CPU this computation is the same for all three quadrature-based tree explainers. Our tree instantiation was developed independently of TreeGrad-Shap (Li et al., 2026), which appeared while this work was under development, and Quadrature-TreeSHAP (Wettenstein et al., 2026) is concurrent; all three evaluate the same leaf-wise polynomial at the same Gauss–Legendre nodes (Section A), the telescoping simplification applies to each, and they differ only in the node budget (Quadrature-TreeSHAP fixes eight nodes, the other two use the exact count) and in implementation.

Table 5: Tree explainer benchmark: runtime (milliseconds per instance) on synthetic data across leaf counts. Stability of the efficiency axiom $\max_i |\mathbb{E}[f] + \sum_j \phi_{ij} - f(x_i)|$: ● exact ($<10^{-12}$), ● degraded (10^{-12} to 10^{-4}), ● broken ($>10^{-4}$). Best per column **bold**; **T/O** denotes timeout after 300s.

#leaves	$d = 10$					$d = 100$				
	10	100	1k	10k	100k	10	100	1k	10k	100k
TREESHAP	0.009 ●	0.087 ●	1.2 ●	14 ●	169 ●	0.010 ●	0.088 ●	1.4 ●	22.1 ●	254 ●
FASTTREESHAP v1	0.010 ●	0.049 ●	0.68 ●	7.6 ●	89.1 ●	0.010 ●	0.055 ●	0.70 ●	10.3 ●	130 ●
FASTTREESHAP v2	0.040 ●	0.11 ●	2.4 ●	7.6 ●	88.9 ●	0.038 ●	0.16 ●	3.7 ●	10.6 ●	123 ●
LINEAR TREESHAP	0.015 ●	0.050 ●	0.48 ●	6.1 ●	69.1 ●	0.016 ●	0.050 ●	0.49 ●	5.9 ●	66.5 ●
SHAPIQ	3.3 ●	53.8 ●	573 ●	6281 ●	T/O	3.4 ●	47.0 ●	576 ●	4602 ●	46494 ●
QUADRASHAP	0.005 ●	0.030 ●	0.29 ●	3.7 ●	41.5 ●	0.006 ●	0.031 ●	0.33 ●	4.0 ●	48.3 ●

The stability that quadrature brings over interpolation-based coefficient recovery is therefore shared by all three, and a CPU comparison among them would compare implementations of one algorithm; the CPU benchmark below is accordingly against the established tree explainers. On the GPU the implementations differ substantially. TreeGrad-Shap has no GPU implementation, and we compare our native CUDA implementation with the CUDA backend of Quadrature-TreeSHAP in XGBoost and with GPUShap (Mitchell et al., 2022); since the first two evaluate the same quadrature representation, the differences between them measure the CUDA implementations rather than the method.

E.3 CPU BENCHMARK

We compare against SHAP (Lundberg & Lee, 2017), FASTTREESHAP v1/v2 (Yang, 2021), LINEAR TREESHAP (Yu et al., 2022a;b), and SHAPIQ (Muschalik et al., 2024). Numerical correctness is assessed via the worst-case violation of the Shapley efficiency axiom, which is zero by design for any exact implementation.

Synthetic data. We benchmark all tree-based methods on random forests trained on a controlled synthetic regression dataset (see Appendix F for construction details), sweeping $d \in \{10, 100\}$ and ensemble size from 10 to 10^5 leaves. To assess algorithmic stability, we use $\max_i |\mathbb{E}[f] + \sum_j \phi_{ij} - f(x_i)|$ as a proxy, measuring the worst-case deviation from the efficiency axiom. Table 5 summarizes the order-of-magnitude differences in this stability proxy; Table 9 in Appendix H reports the exact values. QUADRASHAP ($m_q = \lceil \eta/2 \rceil$) is the fastest numerically stable method in every configuration. At 10^5 leaves it runs in 41.5 ms ($d=10$) and 48.3 ms ($d=100$), outperforming FASTTREESHAP v1 by 2–2.7 $\times$ and SHAP by 3–5 $\times$, with the gap widening as the ensemble grows. Its efficiency-axiom violation remains at floating-point precision ($\sim 10^{-13}$) throughout. LINEAR TREESHAP matches QUADRASHAP in speed but is unreliable: its efficiency violation grows from 6×10^{-14} at 10 leaves to ~ 10 at 10^4 leaves ($d=10$), reaching 7×10^2 at 10^5 leaves ($d=100$). Instability sets in well before the asymptotic regime and worsens with tree depth, making its reported runtime meaningless at scale. SHAPIQ is correct but orders of magnitude slower, timing out at 10^5 leaves for $d=10$.

Real-world text classification. To assess performance on realistic, deep tree structures, we train random forest classifiers on five public text-classification datasets: *emotion* (Saravia et al., 2018), *imdb* (Maas et al., 2011), *sms_spam* (Almeida et al., 2011), *sst2* (Wang et al., 2019), and *rotten tomatoes* (RT) (Pang & Lee, 2005). Documents are encoded as 5,000-dimensional TF-IDF vectors; full training, and dataset details are in Appendix G. The resulting ensembles range from 24k to 181k leaves and reach depths up to 100, substantially deeper than synthetic forests above.

Table 6 reports runtime and summarizes stability differences by order of magnitude; Table 10 in the Appendix gives the exact values. QUADRASHAP is the fastest method on every dataset, achieving 2–4 $\times$ speedups over LINEAR TREESHAP and 15–30 $\times$ over SHAP and FASTTREESHAP. It is also the only method that remains numerically stable across all five datasets: at depth 100, every competitor exhibits catastrophic efficiency violations: SHAP and FASTTREESHAP reach $\sim 10^9$, SHAPIQ reaches $\sim 10^{30}$, and LINEAR TREESHAP reaches 10^{186} – 10^{271} , while QUADRASHAP stays

Table 6: Tree explainer benchmark on five text-classification datasets: runtime (millisecond per instance) with stability of the efficiency axiom shown as ● exact ($<10^{-12}$), ● degraded (10^{-12} to 10^{-4}), ● broken ($>10^{-4}$). Best per column **bold**; T/O denotes timeout after 300s.

	emotion	imdb	sms_spam	sst2	RT
max depth	100	20	100	100	100
total leaves	24,200	51,099	29,662	180,676	83,125
tree count	100	100	100	300	300
TREESHAP	341 ●	27.4 ●	176 ●	1707 ●	773 ●
FASTTREESHAP v1	326 ●	18.4 ●	166 ●	1621 ●	740 ●
FASTTREESHAP v2	326 ●	T/O	166 ●	1618 ●	740 ●
LINEAR TREESHAP	43.2 ●	6.76 ●	10.8 ●	87.4 ●	43.3 ●
SHAPIQ	1650 ●	2205 ●	1930 ●	T/O	T/O
QUADRASHAP	11.3 ●	3.75 ●	9.68 ●	60.5 ●	27.3 ●

at floating-point precision ($\leq 10^{-13}$). The exception is `imdb`, whose shallow forests (max depth 20) fall below the breakdown threshold, and all stable methods agree there.

E.4 GPU BENCHMARK

Experimental setup. We compared the CUDA implementation of QUADRASHAP with three closely related tree explainers: the GPU implementation of Quadrature-TreeSHAP in XGBoost (Wettenstein et al., 2026), the official GPUPtreeExplainer of the SHAP library (Mitchell et al., 2022), and the official TreeGrad-Shap implementation (Li et al., 2026). All methods computed first-order Shapley values under the path-dependent, cover-based tree value function of Section E.1. The primary GPU benchmark was conducted on an NVIDIA RTX A6000. We used the QUADRASHAP repository at commit `3aac1099ac26a59a42acca8f12fdfe6b320cafff`, XGBoost 3.4.1 and SHAP v0.52.0. The XGBoost binary was verified to use its Quadrature-TreeSHAP CUDA backend for `pred_contribs=True`. Only build-compatibility changes were required to compile SHAP for the available CUDA version; the GPUPtreeSHAP algorithm was not modified. Synthetic regression data were generated with target

$$y = \sum_{j=1}^{\min\{8,d\}} [\sin(x_j) + 0.35 x_j^2] + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, 1).$$

The benchmark contained six model configurations: single trees with 16 features and 256, 4,096 or 16,384 leaves; a single tree with 100 features and 4,096 leaves; and forests of 32 trees with 128 leaves per tree and either 16 or 100 features. Tree depth was capped at 24 to remain within the path-size limitation of the released GPUPtreeSHAP implementation, and every requested leaf count was attained. Each configuration was trained with three seeds (42, 43, 44) and evaluated at batch sizes $B \in \{1, 10, 64\}$, giving 18 model–batch cells. The same fitted tree topology, thresholds, leaf values, covers and input observations were supplied to all methods. Conversion to XGBoost introduced a maximum prediction discrepancy of 9.35×10^{-7} , primarily associated with XGBoost’s single-precision tree representation.

Timing protocol. We measured the complete latency of each public explanation call rather than isolated CUDA-kernel time. Every timed call received a fresh contiguous host-array copy, and a fresh XGBoost `DMatrix` was constructed inside every Quadrature-TreeSHAP call; neither input containers nor previously computed explanations were cached. Model training, model conversion, explainer construction and initial CUDA compilation were excluded from the timed region. Every call was followed by explicit GPU synchronization, and numerical CPU libraries were restricted to one thread. For each seed we performed three warm-up calls followed by nine synchronized timed repetitions; the reported time for a model–batch cell is the median of the three seed-level medians and refers to the complete batch, not to an individual observation.

Table 7: GPU tree benchmark: representative median runtime in milliseconds for a complete batch of size B (NVIDIA RTX A6000; lower is better).

Model	B	QUADRASHAP	Quadrature-TreeSHAP	GPUPTreeSHAP
1 tree, $d = 16$, 256 leaves	1	0.410	1.327	2.407
1 tree, $d = 16$, 4,096 leaves	10	1.564	30.620	7.259
1 tree, $d = 16$, 16,384 leaves	10	1.952	121.789	22.307
1 tree, $d = 100$, 4,096 leaves	64	3.156	14.436	8.008
32 trees, $d = 16$, 128 leaves/tree	64	1.426	1.460	5.686
32 trees, $d = 100$, 128 leaves/tree	64	2.119	1.544	6.102

GPU results. The exact, structure-dependent QUADRASHAP implementation was faster than GPU Quadrature-TreeSHAP in 17 of the 18 model–batch cells. The geometric-mean speedup was $5.46\times$ and the largest speedup $62.38\times$, attained for the 16,384-leaf tree at batch size 10. The only Quadrature-TreeSHAP win occurred for the 32-tree, 100-feature forest at batch size 64, where it required 1.544 ms against 2.119 ms for QUADRASHAP, a $1.37\times$ advantage; the corresponding 16-feature forest was approximately tied at this batch size. Against GPUPTreeSHAP, QUADRASHAP was faster in all 18 cells, with speedups from $2.48\times$ to $12.65\times$ and a geometric mean of $5.13\times$. The advantage increased notably for large single trees, where the quadrature formulation avoids the quadratic dependence on path depth of the original TreeSHAP recurrence.

Equal-node control. Because the XGBoost implementation of Quadrature-TreeSHAP uses a fixed budget of eight quadrature evaluations, we conducted a separate control in which QUADRASHAP was also forced to use $m_q = 8$. Both methods used their native eight-point rules, so the number of function evaluations was matched although the transformed nodes and weights were not identical. This experiment used five warm-ups and 25 synchronized repetitions per seed. Under this equal-budget condition QUADRASHAP remained faster in 16 of 18 cells, with a $5.03\times$ geometric-mean speedup and a maximum of $62.83\times$; Quadrature-TreeSHAP won the two batch-64 forest cases by $1.55\times$ and $1.59\times$. The forced-eight QUADRASHAP output differed from its exact output by at most 4.10×10^{-14} on this suite. The principal performance difference therefore cannot be explained by different quadrature-node counts.

TreeGrad-Shap comparison. The official TreeGrad-Shap implementation was evaluated separately on the same six model shapes, three seeds and three batch sizes, with one warm-up and five timed repetitions. Since the released implementation is a single-row NumPy/Python CPU method, it ran on an AMD EPYC 9334 CPU, whereas QUADRASHAP ran on the RTX A6000; for forests, TreeGrad-Shap was applied to each estimator separately and the attributions were averaged, as linearity permits. QUADRASHAP was faster in all 18 cells, with speedups from $9.85\times$ to $3578.68\times$. These numbers compare the available implementations as complete systems and must not be read as an intrinsic algorithmic speedup, because devices, implementation languages and batching strategies differ substantially.

Numerical agreement. In an independent, untimed validation, TreeGrad-Shap and exact QUADRASHAP agreed to within 8.88×10^{-15} . The maximum attribution differences relative to exact QUADRASHAP were 1.79×10^{-6} for Quadrature-TreeSHAP and 7.26×10^{-6} for GPU-TreeSHAP, and exact QUADRASHAP satisfied the efficiency axiom to within 1.42×10^{-14} . The larger discrepancies of the external GPU implementations are consistent with their single- or mixed-precision representations and are not evidence that they compute a different path-dependent game.

Interpretation. The comparison with Quadrature-TreeSHAP does not establish a better asymptotic tree algorithm: both methods evaluate essentially the same quadrature representation. The observed difference is attributable to GPU execution and to the organization of the public call. QUADRASHAP prepares its compact tree representation once, retains it on the GPU, evaluates subtree quantities in levelwise bottom-up and top-down passes, and reduces contributions grouped by feature without atomic updates. The tested XGBoost path reconstructs and transfers its compressed quadrature representation during each public explanation call, schedules work by row and tree with a depth-first traversal, and accumulates contributions with atomic additions. The specialized CUDA tree implementation does not use the generic product-game evaluator of Section 3.2, so the measured tree speedup should not be attributed to it. For GPUPTreeSHAP, both methodology and implementation contribute: quadrature removes the quadratic path-depth factor, while the released GPUPTreeSHAP

wrapper also repeats path construction, transfer, allocation and bin-packing work for each explanation call. The TreeGrad-Shap result is strictly a comparison of available implementations, as no GPU implementation of that method exists. Finally, these experiments used one GPU, synthetic regression forests, depths of at most 24 and batches of at most 64, and they time end-to-end public explanation calls rather than isolated kernels. They support a strong low-latency and large-single-tree result on the tested system, not a claim of unconditional dominance across hardware, datasets or workload shapes.

F SYNTHETIC DATASET CONSTRUCTION

The synthetic regression benchmark uses datasets generated with `sklearn.datasets.make_regression`. For each feature dimensionality, we generate $n=1,000$ samples with the following fixed parameters:

- `n_informative =  $d/4$` (one-fourth of features are informative);
- `n_targets = 1` (single real-valued target);
- `noise = 0.1` (small Gaussian noise added to the target);
- `random_state = 42` (fixed seed for reproducibility).

G TEXT-CLASSIFICATION DATASET DETAILS

All five datasets used in the real-world tree ensemble benchmark (Section 5.2) and the product-kernel benchmark (Section 5.3) are loaded directly from the Hugging Face `datasets` library. Table 8 summarizes their basic statistics.

Table 8: Statistics of the five text-classification datasets used in Sections 5.2 and 5.3.

Dataset	Train	Test	Classes	Task
emotion	16,000	2,000	6	Emotion recognition
imdb	25,000	25,000	2	Sentiment analysis
sms_spam	4,457	1,115	2	Spam detection
sst2	67,349	872	2	Sentiment analysis
rotten_tomatoes	8,530	1,066	2	Sentiment analysis

Text preprocessing. Documents are tokenized with TF-IDF using the following fixed configuration: lower-casing, English stop-word removal, unigram and bigram features (`ngram_range=(1, 2)`), minimum document frequency `min_df=3`, maximum document frequency `max_df=0.95`, sublinear TF scaling (`sublinear_tf=True`), and vocabulary capped at 5,000 terms (`max_features=5000`). This yields a $d=5,000$ dimensional TF-IDF feature vector for each document.

Tree ensemble models. For the tree ensemble benchmark, a `RandomForestClassifier` is trained on each dataset with `n_estimators = 100` (emotion, imdb, sms_spam) or 300 (sst2, rotten_tomatoes), `max_depth = 100`, `min_samples_leaf = 1`, and `random_state = 42`. No hyperparameter search is performed for the tree models; the configuration is fixed across all datasets to ensure a fair stability comparison at large depths.

SVM model. For the product-kernel benchmark (Section 5.3), a support vector classification model with RBF kernel is trained on each dataset. Hyperparameters C and γ are tuned independently per dataset using Optuna with 50 trials of TPE sampling and five-fold cross-validation on the training split. The best configuration is then re-fitted on the full training set. Fifty held-out test samples are drawn uniformly at random for the explanation benchmark.

H EXPERIMENTAL RESULTS

Due to space constraints, the full benchmark tables for tree models are presented here in the appendix. We report the synthetic and real-world tree ensemble results (Tables 9 and 10). Each table reports

runtime alongside the worst-case violation of the Shapley efficiency axiom $\max_i |\mathbb{E}[f] + \sum_j \phi_j(x_i) - f(x_i)|$, which equals zero for any numerically exact implementation; **red** entries indicate numerical breakdown. All benchmarks were performed on a computer equipped with AMD Ryzen AI 7 350 operating under normal conditions and having 32GB of RAM having frequency of 3750 MHz and operating in dual-channel mode.

Table 9: Single-threaded TreeSHAP runtime (milliseconds, mean $\pm$ sample std. over 3 seeds, top) and worst-case efficiency-axiom violation $\max_i |\mathbb{E}[f] + \sum_j \phi_{ij} - f(x_i)|$ (bottom) on synthetic data. **Red**: numerical breakdown. **Bold**: fastest stable.

#leaves	10	20	50	100	1k	10k	100k
$d = 10$							
TREESHAP	0.009 ± 0.000 $5 \cdot 10^{-14}$	0.016 ± 0.001 $6 \cdot 10^{-14}$	0.036 ± 0.003 $1 \cdot 10^{-13}$	0.087 ± 0.002 $1 \cdot 10^{-13}$	1.2 ± 0.1 $3 \cdot 10^{-13}$	14.0 ± 1.0 $1 \cdot 10^{-12}$	169 ± 13 $3 \cdot 10^{-12}$
FASTTREESHAP v1	0.010 ± 0.004 $5 \cdot 10^{-14}$	0.013 ± 0.000 $9 \cdot 10^{-14}$	0.026 ± 0.001 $9 \cdot 10^{-14}$	0.049 ± 0.105 $1 \cdot 10^{-13}$	0.68 ± 0.07 $3 \cdot 10^{-13}$	7.6 ± 0.7 $1 \cdot 10^{-12}$	89.1 ± 7.7 $3 \cdot 10^{-12}$
FASTTREESHAP v2	0.040 ± 0.027 $3 \cdot 10^{-14}$	0.043 ± 0.002 $6 \cdot 10^{-14}$	0.062 ± 0.001 $6 \cdot 10^{-14}$	0.11 ± 0.00 $1 \cdot 10^{-13}$	2.4 ± 0.4 $2 \cdot 10^{-13}$	7.6 ± 0.3 $1 \cdot 10^{-12}$	88.9 ± 2.2 $3 \cdot 10^{-12}$
LINEAR TREESHAP	0.015 ± 0.005 $6 \cdot 10^{-14}$	0.019 ± 0.001 $2 \cdot 10^{-13}$	0.028 ± 0.000 $8 \cdot 10^{-12}$	0.050 ± 0.004 $9 \cdot 10^{-11}$	0.48 ± 0.01 $3 \cdot 10^{-6}$	6.1 $\pm$ 0.1 1$\cdot$10¹	69.1 $\pm$ 0.6 2$\cdot$10¹
SHAPIQ	3.3 ± 0.0 $2 \cdot 10^{-13}$	7.9 ± 0.6 $3 \cdot 10^{-13}$	25.1 ± 1.0 $3 \cdot 10^{-13}$	53.8 ± 3.2 $1 \cdot 10^{-12}$	573 ± 10 $1 \cdot 10^{-11}$	6281 ± 26 $9 \cdot 10^{-11}$	T/O
QUADRASHAP	0.005 $\pm$ 0.000 $4 \cdot 10^{-14}$	0.009 $\pm$ 0.000 $6 \cdot 10^{-14}$	0.017 $\pm$ 0.000 $6 \cdot 10^{-14}$	0.030 $\pm$ 0.000 $1 \cdot 10^{-13}$	0.29 $\pm$ 0.03 $2 \cdot 10^{-13}$	3.7 $\pm$ 0.4 $5 \cdot 10^{-13}$	41.5 $\pm$ 0.6 $8 \cdot 10^{-13}$
$d = 100$							
TREESHAP	0.010 ± 0.001 $9 \cdot 10^{-14}$	0.017 ± 0.002 $7 \cdot 10^{-14}$	0.042 ± 0.005 $1 \cdot 10^{-13}$	0.088 ± 0.017 $2 \cdot 10^{-13}$	1.4 ± 0.2 $6 \cdot 10^{-13}$	22.1 ± 1.5 $2 \cdot 10^{-12}$	254 ± 29 $6 \cdot 10^{-12}$
FASTTREESHAP v1	0.010 ± 0.001 $6 \cdot 10^{-14}$	0.014 ± 0.001 $1 \cdot 10^{-13}$	0.027 ± 0.003 $1 \cdot 10^{-13}$	0.055 ± 0.009 $2 \cdot 10^{-13}$	0.70 ± 0.08 $7 \cdot 10^{-13}$	10.3 ± 0.9 $2 \cdot 10^{-12}$	130 ± 19 $6 \cdot 10^{-12}$
FASTTREESHAP v2	0.038 ± 0.001 $6 \cdot 10^{-14}$	0.044 ± 0.001 $6 \cdot 10^{-14}$	0.073 ± 0.010 $6 \cdot 10^{-14}$	0.16 ± 0.04 $2 \cdot 10^{-13}$	3.7 ± 0.8 $6 \cdot 10^{-13}$	10.6 ± 0.7 $2 \cdot 10^{-12}$	123 ± 13 $6 \cdot 10^{-12}$
LINEAR TREESHAP	0.016 ± 0.000 $6 \cdot 10^{-14}$	0.021 ± 0.000 $2 \cdot 10^{-13}$	0.029 ± 0.001 $1 \cdot 10^{-12}$	0.050 ± 0.004 $1 \cdot 10^{-10}$	0.49 ± 0.04 $2 \cdot 10^{-5}$	5.9 $\pm$ 0.2 6$\cdot$10⁰	66.5 $\pm$ 1.3 7$\cdot$10²
SHAPIQ	3.4 ± 0.4 $2 \cdot 10^{-13}$	6.8 ± 2.4 $7 \cdot 10^{-14}$	21.1 ± 4.3 $2 \cdot 10^{-12}$	47.0 ± 4.6 $3 \cdot 10^{-12}$	576 ± 35 $3 \cdot 10^{-9}$	4602 ± 22 $9 \cdot 10^{-5}$	46494 ± 510 $1 \cdot 10^{-4}$
QUADRASHAP	0.006 $\pm$ 0.000 $6 \cdot 10^{-14}$	0.009 $\pm$ 0.000 $6 \cdot 10^{-14}$	0.017 $\pm$ 0.001 $2 \cdot 10^{-13}$	0.031 $\pm$ 0.002 $1 \cdot 10^{-13}$	0.33 $\pm$ 0.01 $2 \cdot 10^{-13}$	4.0 $\pm$ 0.3 $3 \cdot 10^{-13}$	48.3 $\pm$ 2.8 $5 \cdot 10^{-13}$

Table 10: Execution time in milliseconds (mean $\pm$ sample std. over 3 repeats, top) and worst-case violation of the efficiency axiom (bottom) for tree explainers across text-classification datasets. **Red** entries indicate numerical breakdown. **Bold**: fastest stable.

	emotion	imdb	sms_spam	sst2	RT
max depth	100	20	100	100	100
total leaves	24200	51099	29662	180676	83125
tree count	100	100	100	300	300
TREESHAP	341 $\pm$ 1 $2 \cdot 10^9$	27.4 $\pm$ 0.2 $3 \cdot 10^{-14}$	176 $\pm$ 1 $1 \cdot 10^9$	1707 $5 \cdot 10^8$	773 $\pm$ 2 $1 \cdot 10^9$
FASTTREESHAP v1	326 $\pm$ 0 $1 \cdot 10^9$	18.4 $\pm$ 0.0 $3 \cdot 10^{-14}$	166 $\pm$ 0 $3 \cdot 10^9$	1621 $6 \cdot 10^8$	740 $\pm$ 1 $6 \cdot 10^8$
FASTTREESHAP v2	326 $\pm$ 0 $1 \cdot 10^9$	T/O	166 $\pm$ 0 $3 \cdot 10^9$	1618 $6 \cdot 10^8$	740 $\pm$ 1 $6 \cdot 10^8$
LINEAR TREESHAP	43.2 $\pm$ 0.1 $5 \cdot 10^{222}$	6.76 $\pm$ 0.32 $1 \cdot 10^{23}$	10.8 $\pm$ 0.0 $1 \cdot 10^{186}$	87.4 $\pm$ 0.4 $2 \cdot 10^{268}$	43.3 $\pm$ 0.1 $9 \cdot 10^{235}$
SHAPIQ	1650 $1 \cdot 10^{30}$	2205 $2 \cdot 10^{-8}$	1930 $4 \cdot 10^{29}$	T/O	T/O
QUADRASHAP	11.3 $\pm$ 0.0 $6 \cdot 10^{-15}$	3.75 $\pm$ 0.02 $1 \cdot 10^{-13}$	9.68 $\pm$ 0.01 $3 \cdot 10^{-14}$	60.5 $\pm$ 0.2 $4 \cdot 10^{-13}$	27.3 $\pm$ 0.0 $1 \cdot 10^{-13}$